

Balancing Nutrition and Sustainability

A Tool for informed Dietary Choices

Luna Annabell L. Jensen, Sofie Fog, Jeppe K. Andersen and Jacob
A.L. Jacobsen

Group 5, December, 2024.

DAD7-P7



Titel: Balancing Nutrition and Sustainability: A Tool for informed Dietary Choices

Theme: P7

Project period: 1st October 2024 -
20th December 2024



AALBORG UNIVERSITET
STUDENTERRAPPORT

Project group 5

Luna Annabell Linné Jensen
lalj21@student.aau.dk

Jeppe Kjærgaard Andersen
jeppea21@student.aau.dk

Sofie Fog
sfog21@student.aau.dk

Jacob Andreas Lund Jacobsen
jalj19@student.aau.dk

Supervisor: Dimitrios Raptis

Institute: Institut for Datalogi

Hand in: 19. Dec. 2024

Deadline: 20. Dec. 2024

Abstract

In a time where diet and climate change are increasingly linked, young Danes face the problem of making informed dietary choices, that are both nutritional and climate friendly. This project aims to create a web based solution, that is capable of informing you people about their foods nutritional profile and their CO2 emission. Based upon literature and extensive surveys it became apparent that young Danes are willing to eat in a more sustainable manner, but they lack the knowledge to do so. Our answer makes use of related literature, computational thinking and previous digital tools to design a web solution capable of giving the users both precise nutritional and emissions feedback on the food they consume. The web solution features visualisation, comparison, personal and user friendly nutrition and emissions feedback. Our web solution gives young Danes a tool, which they can use to track and improve their nutritional intake while reducing their CO2 emissions.

Pages: 50

Appendix:0

Contents

1	Introduction	4
2	Related works	7
2.1	Healthy diet from sustainable food	7
2.2	Linear programming and nutrition	8
2.3	Applications	10
2.3.1	MyFitnessPal	10
3	Method	13
3.1	Computational Thinking	13
3.1.1	Decomposition	13
3.1.2	Pattern recognition	14
3.1.3	Abstraction	15
3.1.4	Algorithmic Design	16
3.2	Requirements	17
4	Construction	20
4.1	Introduction to construction	20
4.1.1	Tools other than programming languages	20
4.1.2	Frontend development	21
4.1.3	Backend development	21
4.1.4	Structure of program	22
4.1.5	General repeated codes	24
4.2	Database	25
4.2.1	Construction of createtables.py	27
4.3	Requirements	30
4.3.1	Requirement 1 - User Login	30
4.3.2	Requirement 2 - Search food item & calculation of macros & CO2	33
4.3.3	Construction of remove button	36
4.3.4	Requirement 3 - Display macros & CO2 emissions	37
4.3.5	Requirement 4 - Display Results	40
4.3.6	Requirement 5 - Transparency	42

4.4	Design choices	43
5	Discussion	45
5.1	Klimadatabasen	45
5.2	CONCITO Survey	46
5.3	Dietary recommendations	47
5.4	Requirements	47
5.5	Further work	50
6	Conclusion	52
7	Reflection	53
	Bibliography	53

1 Introduction

In today's world, where environmental sustainability and ethical considerations are gaining prominence, the challenge lies in balancing these concerns with the fundamental need for proper nutrition. As young consumers face an overwhelming amount of information about food choices, it becomes essential to recognize that nutrition should not be overshadowed by environmental or ethical factors. An article from the European Union encourages young people to consider the following things, when it comes to grocery shopping: limit waste, support community-based agriculture, look after environmentally friendly products and go after ethical products (Union 2021). This article demonstrates the sea of information that young people have to navigate when it comes to their choice in food. A sea where some important aspects may drown. Critical consumerism is on the rise and the aspects the report mentions are valid, it is also necessary to prioritise nutrition. Nutrition concerns "the action or process of supplying, or of receiving, nourishment or food (Dictionary 2024). Nutrition directly correlates to a consumer's choice of food and here critical consumerism plays a role alongside challenges such as available budget, knowledge about products and the products impact on the climate (Lawo et al. 2021, p. 13).

Many young Danes struggle with planning meals and cooking (Thingsted 2021). This may negatively affect their nutrition, which plays a crucial role in overall health and prevention of diet related illnesses. In "Danskernes sundhed – Den Nationale Sundhedsprofil 2021", it is noted that there has been a rise in unhealthy dietary patterns amongst Danes in the last decade (Sundhedsstyrelsen 2021, p. 12). Danes scarcely meet the suggested nutritional recommendations made by Fødevarestyrelsen according to the report. For instance, more than nine out of ten young Danes do not eat enough fruit and vegetables a day (Sundhedsstyrelsen 2021, p. 96). However, this specific demographic has an aspiration to eat healthier, whilst they also aspire to be more climate friendly. In the recent years, young Danes have become increasingly aware of climate change (Clara Laurine Berg-Jensen 2021, p. 5).

While critical consumerism emphasises the importance of considering the environmental impact of our food choices, the level of CO₂ emissions depends on the specific country or region. Production and consequently emissions, depends on different factors, such as local production, transportation, and the resources available in each specific country. In Denmark, the average citizen emits approximately 13 tons of CO₂ annually, with around 20% of these emissions stemming from food consumption, which is nearly double the global average emission of around 6.5 tons per person (Minter and Jensen 2024). However, studies show that Danes are generally willing to adjust their choice in nutrition in support of climate action. More than half of the Danish population

consider sustainability when purchasing food. (Madkulturen 2019) Nearly a quarter, 24%, actively choose organic products when shopping while 70% of Danes agree that adjusting daily food consumption is essential to addressing climate challenges (Madkulturen 2019).

As mentioned earlier, young Danes consider climate change to be a very serious issue. About 70% of the participants between 18-29 in a survey by CONCITO reported a significant concern about climate changes, a sentiment shared by only 50% of participants aged 40-49 (Clara Laurine Berg-Jensen 2021, p. 5). Despite the awareness Danish youth possess regarding climate change, their dietary choices do not reflect their climate concerns. However, a significant issue arises - half of Danes report that they lack adequate knowledge about which foods could be considered sustainable (Madkulturen 2019, p. 6). The gap in knowledge directly affect their ability to make informed choices, even when having the intention of buying more climate friendly food. Danes are generally willing to adjust their diet towards climate friendly food, but the lack of knowledge hinders half of the population. As an example, the typical diet of a Dane includes an average of 52 kilos of meat and poultry in a year (Miriam 2024). Only 8 of the 52 kilos comes from poultry, which is problematic because red meat such as beef, has higher CO₂ emissions than poultry. It is not however only a matter of meat choices, that affects individuals CO₂ emissions; general knowledge about the environmental impact of all food choices plays a crucial role in making sustainable decisions.

Small changes to individuals' dietary choices can lead to significant lower CO₂ emissions, and it is evident that targeting the younger demographic for dietary changes is particularly effective, as this group tends to be more receptive to change compared to other age groups (Thingsted 2021). Thus, this demographic is particularly relevant in working with the connection between CO₂ emissions and nutrition. Essentially, fossil fuels and our network of global food systems are closely intertwined as CO₂ emissions follows almost every step of the process that brings food to the table. Being conscious about nutritional intake can help reduce CO₂ emissions and have a positive sway over the looming global warming (Springmann et al. 2016, p. 4146). Young Danes are concerned about climate change and they are aware that nutrition choices directly impact their individual CO₂ emissions. The problem occurs because most young people lack sufficient knowledge to make informed climate-friendly food choices. This ultimately signifies a gap between their intentions and actual consumption habits. To navigate all these different types of criteria and information whilst still staying within a tight budget poses a challenge for many young people. Despite the difficulty it is an important task as nutritional health and climate change positively benefits one another (Springmann et al. 2016, p. 4147). A big part of our future lies in our nutrition and the production thereof, as this will dictate the health of not only ourselves but also the planet (Willett 2020, p. 485). The balance between healthy nutritional food and climate friendly food needs to be made apparent for young Danes, for them to positively impact their health and climate change through their nutrition. We believe it can be achieved by providing a tool that can inform young

Danes regarding both CO₂ emissions and macronutrient content, helping them make informed, sustainable choices that could benefit both their health and the climate. These observations led to the following problem statement:

How can a web solution provide a tool to help young Danes track and improve their nutritional intake while reducing their CO₂ emissions?

2 Related works

To develop a solution for our problem, it is essential to establish a foundation of knowledge regarding previous work in the field. This ensures we avoid duplicating efforts and allows us to draw on the insights and experiences of others. This section explores different kinds of solutions and information regarding the problem, including both academic papers as well as technology that has tackled a similar problem. In this section we examine how research suggests the balance between nutrition and climate friendly food might take place, what kind of tools others have used and what solutions the market currently offers. For young Danes, understanding the balance of macronutrients and the CO₂ emissions of food choices is crucial for making informed, sustainable decisions.

2.1 Healthy diet from sustainable food

The EAT-Lancet Commissions report on healthy diets from sustainable food systems provides a foundation for understanding the intertwined challenges of environmental sustainability and nutrition. The report emphasises how current food production and consumption practices are inherently unsustainable, contributing to significant environmental damage, including nearly a third of global CO₂ emissions (Willett 2020, p. 449). The report identifies food production as a major source of CO₂ emissions, detailing how a shift in dietary patterns is necessary to sustain both human health and the environment. The commission describes how shifting dietary patterns globally can be done through developing a “reference diet”, which takes emission resulting from food production and the nutritious needs into account (Willett 2020, p. 447). The diet emphasises plant-based foods, such as vegetables, fruits, whole grains, legumes, nuts and unsaturated oils, while including a limited amount of animal-based products like red meat and poultry, which has a particularly high CO₂ emissions (Willett 2020, p. 447). Following these specific diets, countries could achieve reductions in CO₂ emissions.

The report underscores how consumer education and knowledge regarding food production is essential to achieve sustainable food systems. There is a growing concern and awareness amongst consumers globally, however, many consumers still lack the necessary information to make more environmentally conscious dietary decisions (Willett 2020, p. 449). The report advocates for educational interventions and accessible tools that can foster understanding regarding the environmental impacts of different foods for consumers. The Commission concludes that sustainable changes to global food systems are essential to protect the environment. The report

sets boundaries for when food production can be considered sustainable, creating six indicators for when deeming any kinds of food sustainable; land use, freshwater use, biodiversity loss, CO₂ emission and nitrogen and phosphorus cycles (Willett 2020, p. 447). Delaying action increases the risk of failing to meet the UN Sustainable Development Goals and the Paris Agreement targets, concluding how the “reference diet” is a necessity to ensure long-term sustainability regarding food production. Here the problem of how people can balance a nutritious and environmentally sustainable diet arises. A tool which has been used to solve this previously is Linear Programming (LP).

2.2 Linear programming and nutrition

Linear programming (LP) is a mathematical optimisation that has become a powerful tool for formulating nutritionally adequate, environmentally sustainable and culturally appropriate diets (Frega et al. 2012, p. 228). This approach has seen increasing applications in the fields of food and nutrition, enabling researchers to identify dietary solutions that balance health, ecological impact, price, and cultural preferences (Masino et al. 2023, p. 2).

LP helped optimise food choices while preserving cultural food practices, proving effective in aligning health and climate goals in Germany’s diverse dietary landscape. The research conducted in Denmark, Finland, and France approached LP as a tool to inform consumers on climate-friendly diet choices. In particular Danish, Finnish and French models showed how LP could support public health by providing specific evidence-based dietary guidelines (Irz et al. 2019, p. 169). This helps consumers understand the types of foods that meet sustainability criteria, leading to more informed food choices. LP’s mathematical foundation makes it an ideal tool for handling complex dietary goals, enabling researchers to explore various trade-offs and set specific constraints based on health guidelines, emissions targets, and food costs. LP-based diet models are highly adaptable allowing for customisation to meet the needs of different demographics, dietary restrictions, and environmental objectives.

The main objective of our project is to create a web solution to help young Danes track and improve their nutrition whilst reducing their CO₂ emissions. This means our focus will be on the users’ individual inputs concerning their food intake. However, we can use LP to give our users specific feedback involving the factors CO₂ and calories as well as macro nutrients. LP offers a scientifically grounded and mathematically rigorous approach to assist users in visualising their dietary patterns. This capability positions LP as a vital instrument in the global endeavour to promote sustainable diets. LP is also capable of making models possible as well as charting different numbers in relation to one another for example CO₂ emissions in relation to calorie consumption (Benvenuti and De Santis 2020, p. 2). However when considering sustainable eating patterns it is also necessary to consider potential biases in research. This will be explored in the following section.

Dietary recommendations and industry influence

Changes in lifestyle often lead to shifts in eating patterns, with convenience frequently taking precedence over nutritional needs, particularly among young people. In response to the growing focus on climate, there has been a shift in food recommendations. An increasing number of advertisements now promote plant-based options and emphasise healthier alternatives.

Despite this, there are certain issues that become prevalent when looking at nutritional food. Professor Marion Nestle describes in the book "Unsavoury Truth: How Food Companies Skew the Science of What We Eat" how food industries have an influence on what we eat (Nestle 2018, p.74). It is important to note that there is a higher regulation in Denmark compared to the US but themes described in the book are still relevant. In particular, aspects of public awareness and being critical of recommendations seem relevant in the Danish perspective, not just in the US. The food industry often works with advertising benefits, which may be misleading. An example of this is the advertisements of plant-based alternatives. Plant-based options often are made to seem better than animal-based products, in terms of advertising low-fat, high protein and CO₂ reduction. However research conducted by "Forbrugerrådet tænk", indicate that there are more things to consider. "Er plantebaserede alternativer mere forarbejdede end kød?" (Andresen 2023) looks at the ingredients and nutritional value between the two categories, plant-based and animal-based. They concluded that there are multiple factors for and against both groups. In one instance they look at "hønsesalat", which shows that the plant-based version is less processed and that there are more E-numbers in the meat version, which would suggest plant-based food is better. When looking at plant-based cheese versus milk-based cheese, it is from a nutritional perspective a bad alternative to choose the plant based product, as there is a high content of saturated fat and no protein compared to the milk-based cheese. Thus choosing plant-based foods over animal-based products will not necessarily indicate healthier or better alternatives. One of the experts Astrid Marie Lauridsen suggests looking at content of fat and saturated fat, salt and sugar in terms of better choices rather than looking at animal-based or plant-based (Andresen 2023).

Nestle's critical perspective on the food industry emphasises the importance of transparency and scepticism in understanding nutritional information (Nestle 2018, p.118). These insights are valuable for both consumers and developers. For consumers, Nestle's work encourages informed and independent choices, helping them navigate the food industry's marketing and research. For developers working with food-related guidelines and technologies. Nestle's perspective offers a framework for building transparent, evidence-based tools that consumers can trust (Nestle 2018, p.118). Applying these insights can promote a healthier and more informed approach to nutrition for both developers and consumers.

The related works section demonstrates a rather challenging problem of eating sustainable and nutritious. Not only is it difficult to balance in everyday life but the complexity food industry makes even more difficult to make the informed choices. Having a sustainable diet requires adequate

information regarding what makes a diet sustainable and what kind of tools, such as LP, makes this process easier. Our findings from the related works not only expands our understanding of the problem area, but it also enables us to set up a list of requirements for our solution.

2.3 Applications

To get an idea of what functionalities would be practical, we looked at apps already on the market in the fields of nutrition and CO₂ emissions. To explore this, we examine the apps MyFitnessPal and Floop, which offered insights into the features available and how they are implemented in the development of similar projects. The two apps are simple in their structure and how users are encouraged to use them. In both apps the user logs food in order to receive further information on their eating patterns. The Floop app is an example of an app working with CO₂ emissions regarding food. The app has two main aspects, which is the “Your dietary climate impact” and “This week’s meals” functions. The first involves showcasing the logged food items in a different way than just providing the summarised calculation of the user’s dietary CO₂ emissions. The other provides an option for users to select food items and plan meals ahead of time. Due to the similarity in the two apps, we will go further into depth with MyFitnessPal, as it provides almost the same features but with a different perspective, while also having more features. The app is widely used, showing successful approaches to working with nutrition.

2.3.1 MyFitnessPal

MyFitnessPal (MFP) is a prominent mobile application and online platform that supports users in monitoring their dietary habits and physical activities, primarily focusing on weight management and overall health improvement. Since its launch in 2005, it has gained immense popularity due to its user-friendly interface and robust features that simplify calorie counting, nutrient tracking, and exercise logging. Central to its appeal is an extensive food database containing millions of items, which makes dietary self-monitoring accessible and efficient for a diverse range of users. Studies have highlighted MyFitnessPal’s role in facilitating healthier lifestyles by providing actionable insights into dietary and physical activity patterns (Al-Hassan 2016, p.64-65), (Gioia et al. 2023, p.12-13). The widespread adoption of MyFitnessPal is attributed to its ability to blend functionality with simplicity. By integrating detailed logging capabilities and visual tools, it caters to individuals aiming monitor their diet.

A part of the core features in MyFitnessPal is the app’s comprehensive calorie and nutrient tracking system. The app makes it possible for users to log meals and beverages in an easily accessible manner through the app’s extensive food database, which accommodates both home-cooked, packaged foods and ingredients in general. Allowing users to quickly access nutritional details for pre-packaged items as well as non-prepacked foods.

One of MyFitnessPal's main features is the ability to let users receive detailed nutritional breakdowns for their logged meals, including macronutrients such as carbohydrates, proteins, and fats, as well as micronutrients like fiber, sugar, vitamins, and minerals. While research supports the app's accuracy in reporting total caloric and macronutrient intake, certain limitations arise from incomplete data within its user-generated database, particularly concerning nutrients like sodium and cholesterol (Al-Hassan 2016, p.52-56), (Gioia et al. 2023, p.12-13). MyFitnessPal also offers tools for tracking progress, including weight logs and detailed activity summaries. Visual representations such as graphs and charts provide users with insights into their journey over time, fostering a sense of achievement and motivating continued effort.

Extensive research has underscored the efficacy of MyFitnessPal as a dietary and weight management tool. A pilot study comparing MyFitnessPal with a 7-day food diary revealed minimal differences in nutrient reporting, affirming the app's viability as an alternative for dietary assessment (Al-Hassan 2016, p.47-52). By automating calorie and nutrient calculations, the app addresses traditional barriers to self-monitoring, making it easier for users to maintain consistent tracking.

Despite its strengths, MyFitnessPal is not without limitations. One significant challenge lies in the reliance on user diligence for accurate logging. Self-reported data can be prone to inaccuracies, whether due to under-reporting, overestimation, or errors in portion sizes. Additionally, while the food database is extensive, it includes user-generated entries that may lack verification, which might lead to inconsistencies in nutrient data. However even in smaller non-user databases inconsistencies can occur (Gioia et al. 2023, p.9-11). These limitations highlight the need for continuous improvement in database accuracy and user education to maximise the app's effectiveness. Nevertheless, MyFitnessPal's convenience and adaptability have cemented its position as a leading tool for health-conscious individuals. The app lets users gain greater control over their dietary habits.

The rise of self-tracking apps like MyFitnessPal reflects a broader shift towards technology-driven health management. Academics have explored the psychological and social dimensions of self-tracking, noting both its benefits and potential drawbacks. On the positive side, apps like MyFitnessPal empower individuals by offering real-time insights into personal habits. This empowerment encourages users to make informed decisions that align with their health goals. When motivating it is important to make sure motivation does not lead to obsession. Striking a balance between motivation and over-reliance on data is an area of ongoing research (Turner 2019, p.43-45). As the field of digital health evolves, understanding these dynamics will be crucial in designing tools that support long-term well-being without unintended psychological harm.

MyFitnessPal exemplifies the potential of technology to transform dietary choices, offering an accessible and effective platform for self-monitoring. The user-friendly features and extensive database make it particularly appealing to individuals seeking to improve their health. Floop works with the same aspects but within dietary CO₂ emissions, rather than the focus on nutrition, which highlights the potential to address environmental concerns alongside dietary tracking. With societies increasing interest in health and environmental impact, an app providing a nutritional

management tool as well as a CO₂ management tool could be an invaluable resource for individuals striving to align personal nutritional goals with sustainable living practices.

3 Method

Before we started developing a web solution to solve our problem statement, we needed to reach an understanding of what the problem concerns and break it down into more manageable components. To not only define the problem but also identify the necessary features for our web solution, we applied the method of Computational Thinking (CT).

3.1 Computational Thinking

CT served as a central framework for developing our web solution. CT involves four key components: decomposition, pattern recognition, abstraction and algorithm design ([BBC 2024d](#)). These principles guided the problem-solving process, from understanding the users needs to implementing the functional features in the web solution. The section below outlines how CT was applied during the development of the project.

3.1.1 Decomposition

Decomposition involves breaking down a complex problem into smaller and more manageable tasks ([BBC 2024c](#)). We began by brainstorming the broad challenge of helping users track their nutritional intake and their CO₂ emissions. Our intention is to improve users' nutritional balance and reduce their CO₂ emissions. With our initial investigations of features from MyFitnessPal and Floop, we further identified sub-tasks such as integrating food databases, designing a user-friendly system, calculation of CO₂ emissions and macronutrients for logged items. These elements were then decomposed further, which led to the problem being divided into manageable sub-problems.

The first being requirements, which included what key components the web solution should possess. The key components include the search function, nutritional CO₂ calculations, and visualisations of the result. The user should be able to use the search function to make a personalised list of food items. The selected items would then be used to calculate the user's CO₂ emission and macro breakdown and shown to the user through simple visualisations. The second is database management. The structure of the database is important to create a functional database, for storing CO₂ and nutrition data for each food item and keeping track of the users in session. This includes organising the food and user data effectively to make retrieval of the data easy when it is needed. The users should be able to use the web solution easily, therefore the third aspect is the frontend design. This involved creating an intuitive interface that is user-friendly and with an appealing

visual layout to ensure a smooth user experience. The backend logic problem involves handling the dynamic calculations and data processing. This includes calculating the nutritional content and the corresponding CO₂ emissions based on the user's selected food items. The final important aspect involves displaying the data through visualisation. The data insights should be presented through interactive charts, enabling the user to see their macro breakdown and their CO₂ emission based on their selected food items. Decomposition allowed us to prioritise and structure our work, ensuring each sub-problem addressed a critical aspect of the problem statement.

3.1.2 Pattern recognition

Pattern recognition involves identifying recurring patterns or similarities within a challenge, which can help optimise solutions (BBC 2024e). While analysing our focus area, we identified multiple patterns in how MyFitnessPal and Floop function and how users interact with them. The first one of these patterns is the user interactions. Users are able to search for food, input their serving and view corresponding nutritional data. Recognising this pattern made us aware of the usability of a similar set up in order to create an intuitive user experience. To test this pattern and possibly expand it in regards to our web solution we have used the Wizard of Oz (WOZ) approach (Bernsen, Dybkjær, and Dybkjær 1994). WOZ prototyping is an innovative and cost-effective approach to product design and development that focuses on testing ideas early in the process. This method involves creating simplified, often non-functional, versions of a product to simulate the user experience and gather feedback before committing to full-scale development. As Bernsen, Dybkjær, and Dybkjær explain (Bernsen, Dybkjær, and Dybkjær 1994, p. 1-2), the term is derived from the use of a "wizard", these "wizards" simulate the use of a product or partial use of one. This creates the illusion of a possible functional product, this illusion makes the one presenting the possible product a "wizard".

Similarly, in WOZ prototyping, a human operator manually simulates a product's functionality, allowing users to experience it as though it were fully or partially functional. This approach bridges the gap between conceptual ideas and practical solutions by enabling designers to refine user experiences based on feedback. For example, a physical block might represent a phone, providing a means to test ergonomic and conceptual features without creating a working prototype. The method contains risks but validates ideas, and ensures resources are directed toward user-centered solutions (Maulsby, Greenberg, and Mander 1993, p.278, 283). This combination of low-cost testing and user feedback makes WOZ prototyping a powerful tool in the early stages of design. We have used WOZ in both pattern recognition and abstraction, by giving each group member a piece of paper simulating the web solution, which we had in the back of our covers on our phones for a week. Afterwards we discussed the patterns and features we found and wanted to implement in our web solution. The pattern of user interaction, which we had already discovered in the apps in related works was further validated and specified through the WOZ approach, we found a similar user

interaction. It also allowed us to recognise additional patterns for creating a better user experience. This includes a data pattern of units as every food item needs to be inserted with a unit of quantity. The quantities of food items should be presented in units appropriate to their form. The ideal for the web solution would thus be a great flexibility in units, for example allowing the user input "100 ml of milk" instead of only accepting "100 grams of milk". However, the important part of the unit pattern to incorporate is consistency of units. This enables us to keep a consistent database, making data processing and retrieval efficient and easy.

3.1.3 Abstraction

Abstraction involves simplifying complex aspects of a problem, ignoring less relevant details (BBC 2024a). The introduction shows the complexity of our problem area, which stresses the importance of abstraction in our project. We have used WOZ in the abstraction process to narrow down the problem area. WOZ prototyping aligns with this principle by simplifying complex systems into manageable parts for testing and validation. In our project, which focused on developing a nutrition-related web solution, WOZ provided a practical means of exploring user needs and refining our concept. The process began with defining the concept. Based on prior research in related works, we identified a gap in existing tools concerning the combination of both food CO₂ emissions and macronutrient tracking. As mentioned in pattern recognition, we made a WOZ prototype which allows us to simulate user interactions and explore potential features without developing the web solution first. This stage required us to consider how the prototype could convincingly mimic a functional product in reality, adhering to the principles of WOZ prototyping outlined by (Bernsen, Dybkjær, and Dybkjær 1994, p. 8-9). Testing took place in real-world contexts, with group members acting as users. Although the method typically involves external participants, time and resource constraints led us to rely on self-testing, which provided valuable insights into usability challenges and feature prioritisation. After the week of us using the WOZ prototype we found the following things vital for our web solution: information about macronutrient content and food emissions, comparable element about the emissions and a search tool for the food.

Recurring themes were identified as high-priority features, while unique ideas were discussed for feasibility. This iterative approach ensured that the final design reflected both shared needs and innovative possibilities. For example, we frequently noted the desire for CO₂ emissions data alongside nutritional information, as well as for a way to compare this with average similar emissions, solidifying these as core elements of the app. The final step involved deciding the project's next direction. Based on the feedback, we determined that a focus on CO₂ emissions and macronutrient tracking offered the most value to users. This combination was less commonly addressed in existing tools and aligned with our goal of providing a unique solution. By abstracting the essential features and testing them through WOZ prototyping, we were able to refine our concept without the costs associated with full-scale development.

WOZ prototyping demonstrates the value of abstraction by enabling designers to isolate and test key functionalities without committing resources to complex systems. This iterative process of testing and refinement allows for identification of user needs and ensures that development efforts are directed toward impactful solutions according to (Bernsen, Dybkjær, and Dybkjær 1994, p. 1-2). WOZ is particularly valuable in fields like software design, robotics, and user experience, where rapid prototyping and user feedback plays a critical role in success (Maulsby, Greenberg, and Mander 1993, p. 278).

In our project, applying WOZ prototyping enabled us to approach abstraction in a practical and user-focused way. By breaking down the problem into manageable components and simulating user interactions, we gained valuable insights into user needs and preferences. This method provided a cost-effective pathway for early-stage development and ensured that our design was grounded in real-world feedback. Through this process, we refined our concept into a viable and innovative solution, combining CO2 emissions tracking with macronutrient analysis. To further streamline the concept, we applied data abstraction by focusing on the most relevant information: nutritional values (kcal, protein, fat, and carbs) and CO2 emissions. Using a CSV file from Klimadatabasen, we abstracted unnecessary columns, making the database lean, manageable, and tailored to our project's needs. The user interface was also abstracted to display only essential outputs, such as total calories, macronutrient breakdown, and CO2 emissions, thereby reducing cognitive load and providing a clear, user-friendly experience. For the backend, functions were designed with a modular approach to handle specific tasks like search, login, and registration. Each function operates independently and can be modified or combined without affecting other parts of the system, enhancing the backend's flexibility, organisation, and maintainability.

3.1.4 Algorithmic Design

Algorithm design is the last step in CT and it is critical for implementing functionalities of the web solution (BBC 2024b). It provides a systematic approach to solving specific problems, to ensure efficiency, clarity and scalability. The design focused on creating a step-by-step procedure. In our web solution we have defined three areas of algorithm design: user authentication, food search and database management, and visualisation tools.

User authentication addresses the need for personalised accounts and secure session management. Through algorithmic design, this functionality is implemented via clear, step-by-step processes that handle tasks such as logging in, maintaining user sessions, and logging out. These algorithms ensure data security and create reliable and efficient user experience. The algorithm design in food search is based upon efficient search algorithms to retrieve data quickly and accurately. Database management algorithms handles adding, removing, and storing user-selected items and ensures data integrity across sessions. The structured, scalable nature of these algorithms supports

the web solution as more user input is added. Visualisation tools also rely on algorithmic design to transform raw data into meaningful insights. By dynamically calculating nutritional values and CO₂ impacts, these tools generate visual representations such as pie charts and scatter charts. These visuals help users understand the macronutrient breakdown of their selected foods and the environmental impact of their dietary choices. We expect these algorithm designs to simplify our construction of the code by ensuring accurate calculations, intuitive interactions, and robust data management.

3.2 Requirements

With the exploration of our problem area in mind and our use of the CT, we have gained extensive insight in our problem area. When designing a web solution it is necessary to consider which requirements there are for the project. Requirements are defined as something a product must be able to do (Benyon 2010, p. 139). To find the requirements we used CT, the WOZ approach and the related works. From this point in our report we are able to make a list of requirements for what makes a successful web solution in regards to our problem statement, see figure 3.1 for requirements and what inspired. In related works it was clear that the apps Myfitnesspal and Floop used personal logins to manage the individual users accounts. Myfitnesspal and Floop also have a search function, where it takes input from users regarding what they have eaten or done. This is also something we came across with the WOZ approach. In MyFitnessPal and Floop it was also possible to set goals, through the WOZ method we viewed this as a possible encouraging function for the users. We aim to not only track the users nutrition and CO₂, but also motivate their goal setting.

As mentioned in the introduction, many young Danes lack knowledge about what kinds of food could be considered climate friendly and with the WOZ method it became clear that we want the users input to be comparable, especially regarding the CO₂ emissions. We plan to include a visual comparison element, allowing each user to compare their individual CO₂ emissions to the average Dane. This will make it easier for the user to relate and understand the magnitude of their emission. This requirement will show the users total macros, CO₂ emissions and it directly relates to our problem statement, which concerns the balance of those two things. Lastly, we want the web solution to be transparent about the information about CO₂, as a way to combat the lack of knowledge about climate friendly foods. We also believe transparency will encourage the users to make it into a habit to track their nutrition and emissions.

Requirement	Description	Inspiration
R1	The system must enable users to have a personal account	Inspiration from the apps Myfitnesspal and Floop
R2	The system should prompt the user for a food item and their quantity in order to calculate macros and CO2	Inspiration from the findings in related works, the WOZ approach and the apps Myfitnesspal and Floop
R3	The system should show the users the total of macros, kcal and CO2 emissions	WOZ approach
R4	The system should show where the user is, compared to an average Danish person.	Inspiration from the findings in related works and the WOZ approach.
R5	The system should show transparency of the application to the user	Inspiration from the findings in related works and the WOZ approach
R6	The system should allow the users to set individual goals their macro intake and CO2 emissions	Inspiration from the apps Myfitnesspal and Floop
R7	Tips on gauging the weight of ingredients	Inspiration from the Floop app. Information regarding the weight of standard ingredients, which could makes it easier for the user, so they do not have to weigh everything all the time.
R8	History element	Inspiration from apps like Myfitnesspal and Floop. The web solution should be able to generate a rapport for macros and CO2 emissions over a longer period of time, so the user can look back and possibly see progress

Figure 3.1: System requirements and their inspirations

CT combined with WOZ built the foundation for solving the problem of tracking nutrition and CO₂. It guided us to grasp the requirements we need to build a user-friendly, scalable and effective web solution. The CT principles ensured that the web solution addresses users' needs while maintaining clarity and efficiency throughout the development process. WOZ gave us an insight in what kinds of features should be implemented to turn the web solution into a useful tool. The requirements will not only be our guidelines for the construction of the web solution, they will also determine and help our evaluation of the entire project. How successful the requirements have been implemented into our web solution or what went wrong with the implementation, will be discussed later in the report.

4 Construction

4.1 Introduction to construction

In the construction of the web solution, programming languages form the foundation for how it processes data and interact with users. They enable developers to write logic, structures and behaviours for the web solution, making it functional and interactive. This section will discuss tools, structure and organisation of code, which will explain the baseline for the section of construction. The tools utilised for this web solution are categorised into three main sections: Frontend (HTML, CSS, and JavaScript), Backend (Python with Flask), and Database (SQLite3). This combination of technologies was chosen for their capabilities, versatility, and compatibility, ensuring seamless integration to create a cohesive and functional web solution(W3Schools 2024d).

4.1.1 Tools other than programming languages

In this project we have used several programming languages, however they are not the only tools, we have used for constructing our web solution. We have received as well as sought inspiration from CS50 and W3. These two tools have provided us with methods that made it easier to construct a log-in feature; for instance, W3 has been particularly helpful in enhancing the frontend development.

Artificial Intelligence (AI) was used when constructing our web solution, specifically when debugging different sections of our code. AI tools like ChatGPT, Copilot and Blackbox helped us identify errors, understanding their causes and provide effective solutions when we encountered bugs or were unable to resolve issues on our own. While the code was written by us, AI tools played a supportive role in helping us navigate through and resolve errors. Our approach ensured that we maintained ownership of the coding process, while leveraging AI to enhance our problem-solving abilities.

4.1.2 Frontend development

The frontend development involves three main tools: HTML, CSS and JavaScript. HTML is a markup language and is used to structure the content of a website. HTML defines the different elements, like headings, paragraphs, links and images. We stored HTML in a folder called templates. In this folder, you can find files for handling the frontpage, information page, search page, registration, login as well as additional pages confirmation and failure messages (W3Schools 2024b). Once the structure of the website is created using HTML, it is important to focus on our visual presentation. This is done through the use of CSS, as it enhances the user experience by styling the HTML elements.

CSS is used for the appearance and layout of the webpage. CSS defines how the HTML files are displayed, thus setting the colours, fonts, sizes and placements of elements on the. This is essential for creating a visually appealing and interactive experience for the user (W3Schools 2024a). While CSS ensures that our webpage is visually appealing, interactivity is added through JavaScript. This is done through the use of dynamic elements, transforming the static design into a more engaging and functional user experience. JavaScript is used to handle webpages, in this web solution it is used for handling charts. JavaScript is responsible for rendering the charts, that are used to display macronutrient breakdown and CO2 emissions data. In this web solution Google Charts has been used to render the charts (W3Schools 2024c).

4.1.3 Backend development

The next component focuses on the backend, which involves server-side processes and interactions. This web solution used Python with Flask as a framework. Python is a widely used high level programming language, known for its simplicity and readability. Flask is a micro framework for Python that provides essential tools for building web solutions, allowing easy handling between routing, requests and responses between frontend and backed (Pallets Projects 2024a). The backend utilises Python and Flask to manage the logic, handle user input, and establish connections with the database.

We use SQLite3 in our web solution to store and manage data. It was also used to create our database, which contains the products from Den Store Klimadatabase.(CONCITO 2024a). We use SQLite3 because it is lightweight and does not require separate servers for database processing and access This works for this web solution as it is still a prototype, and can be ported to a large database once needed (Foundation 2024). This allows the web solution to store user data and other relevant information in a structured way, ensuring efficient data retrieval and management.

4.1.4 Structure of program

The web solution's structure was crucial for organisation and ensuring it aligns with the programming language's capabilities. In order to organise the web solution, folders were used to collect similar files. Therefore, a folder called "Templates" was used to store all HTML files. Another folder called "Static" was used to store images, csv file and CSS files, keeping the static files in a separate folder. Outside these folders are the main file (app.py) used to run the web solution, text file (requirements.txt), and a database file (ozzy.db). The file for creating the tables in the database (createtables.py) was also included. This file is a python file, which will only be run once, creating the tables in the database.

While the folder structure kept the project organised, understanding the flow was equally important for clarity and maintenance. Below is a flowchart(see figure 4.1) that visually represents the web solution's logical flow, including user registration, login, searching for food, and database operations.

The flowchart can be divided into four general categories: User registration and login, Search page, Frontpage, and Information page. The user registration and login is the first part a user is met with when entering the page, here they get options of signing up and logging in. If the user is logged in they will then be directed to the frontpage, from here they are able to choose between the three pages. Each page has specific functions for the web solution. The search page enables the user to create a personal list of food items and the frontpage will visualise the users' macronutritional breakdown and CO2 emissions. The last page functions as an information page focused on the transparency of sources and calculations used for the entire project.

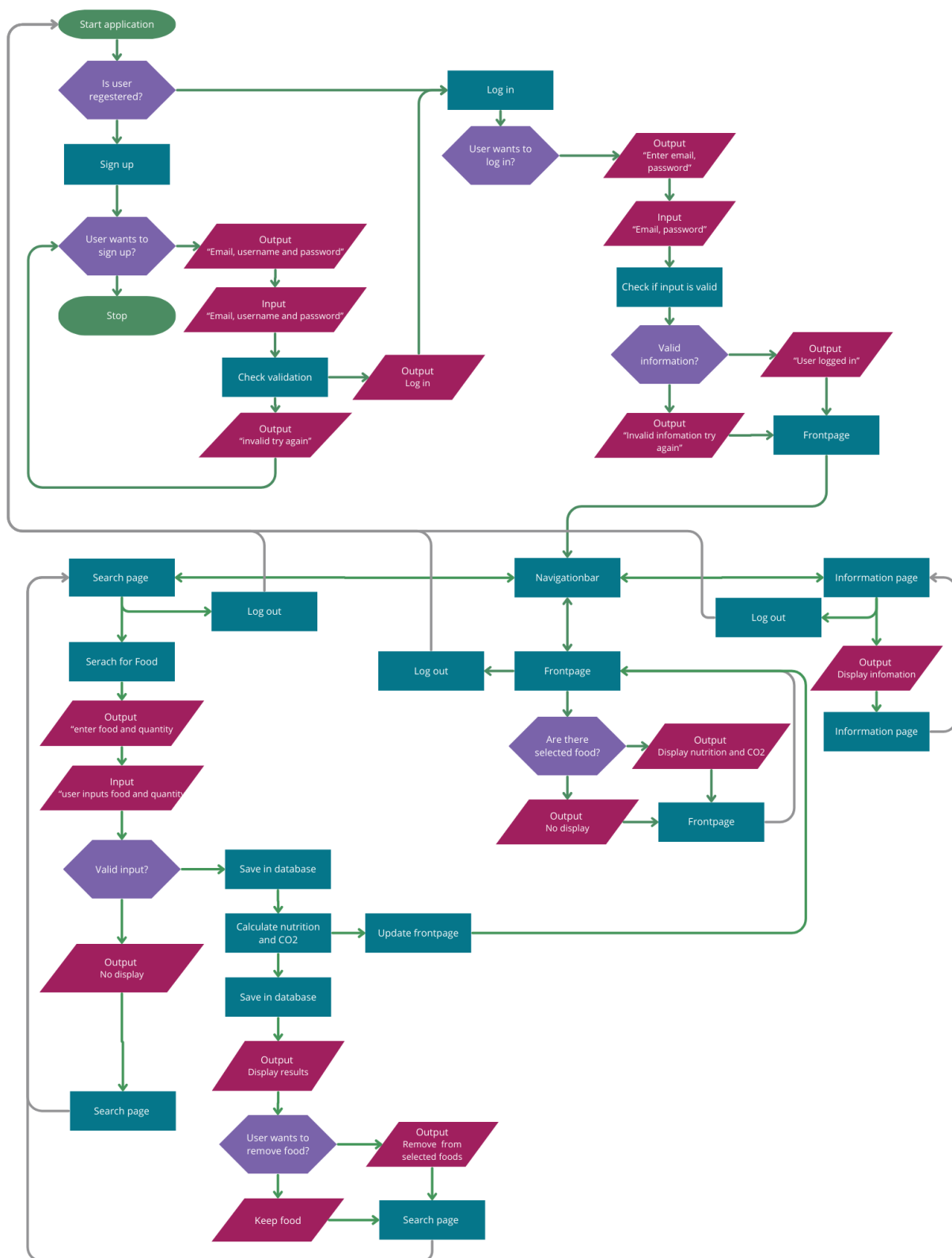


Figure 4.1: Flowchart: Visualising user actions, program logic, and database interaction.

4.1.5 General repeated codes

In our web solution, repeated codes are used across different pages. Here is an explanation of some repeated code snippets. Certain code snippets are repeated throughout the different HTML files, enabling the user to be directed to other pages if interacted with. An example of this is a section used via jinja to handle whether a user wants to log out.

```
{% if Username %}
    <p>You are logged in as {{ Username }}. <a href="/logout
        ">Log out</a>.</p>
{% else %}
    <p style="display:_inline;">You are not logged in. </p>
    <button type="button" onclick="window.location.href='/
        login'">Log in</button>
{% endif %}
```

The section has an if statement that will look for a Username variable and checks if there is a username passed from the flask server. If this is true the line "You are logged in as Username - with username being replaced by the users logged username-will be displayed. It will also provide a link that will display the /logout route (Pallets Projects 2024b). If it is false it will display "You are not logged in." and display a button that when clicked will direct to /login route. Thus, the behaviour is that when the user is logged in the username is stored in the session and passed as Username and when logged out the Username is not defined and the template shows the "not logged in" message. This reflects the user's authentication status in the frontend.

For the navigation bar of the sections are used by simply referencing to the url's for the three sections Frontpage, information page, and search page.

```
<nav>
    <ul>
        <li><a href="{{_url_for('frontpage')}}">Frontpage</a></
            li>
        <li><a href="{{_url_for('information')}}">Information</
            a></li>
        <li><a href="{{_url_for('search')}}">Search</a></li>
    </ul>
</nav>
```

The following snippet works with the </nav> element, which is used to create the navigation bar, as it defines a section containing links. The element creates an unordered list that contains the list of pages to navigate between. The element represents a single list item, in this case, the navigation links. The <a> element defines a hyperlink, allowing the linkage to different pages.

The `url_for` is a Flask function, which automatically replaces `{url_for('information')}` with the actual URL rather than the route `/information` (GeeksforGeeks 2024). The reason for doing it this way rather than directly through the route is that it automatically updates if the route name changes; thus, the route is the only place to update, not everywhere the URL is referenced in the HTML. This makes the solution more robust, flexible, and manageable. This creates navigation code, resulting in a list of links leading to the respective pages within the web solution.

The web solution is structured to be scalable, maintainable, ensuring that it is both functional and easy to manage. The organisation of frontend, backend and database code is key to creating the interactive experience, while repeated code and proper file management make the development process more efficient. The frontend and backend provide interface and logic for the user interaction, these elements depend on a robust and well-integrated database, to store, retrieve and manage data. The database serves as the foundation for the web solution ensuring the interaction between the user and their data runs seamlessly. The following section explains the database design and its role in the web solution.

4.2 Database

To calculate the CO2 emissions and macros of user-inputted food items, we converted a database from "Den Store Klimadatabase" containing CO2 emissions, fats, protein, and carbs of 504 different food items into a CSV file. To avoid reprocessing the CSV file each time the script is executed, the following code was initially active to perform the import but later commented out using `"#"` after the first successful execution. The following section explains how the CSV file is loaded into an SQL table, which is essential for the calculation of CO2, calories and macros.

```
def load_csv_to_db():
    csv_file = 'static/NyDB.csv'
    connection = sqlite3.connect('ozzy.db')
    cursor = connection.cursor()

    with open('static/NyDB.csv', newline='', encoding='utf-8-
sig') as csvfile:
        reader = csv.DictReader(csvfile, delimiter=';')

        for row in reader:
            try:
                if 'ID_Ra' in row and row['ID_Ra']: # Check if
                    'ID_Ra' exists
                    # Insert data into the database
                    cursor.execute("""
```

```

        INSERT OR REPLACE INTO Food_product (
            ID_Ra, Navn, CO2, Energi, Fedt,
            Kulhydrat, Protein)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """
    (
        row['ID_Ra'],
        row['Navn'],
        float(row['CO2']),
        float(row['Energi']),
        float(row['Fedt']),
        float(row['Kulhydrat']),
        float(row['Protein'])
    )
except Exception as e:
    print(f"Error_processing_row_{row}:_{e}")

connection.commit()
connection.close()
print("CSV-filen er blevet behandlet.")

```

Initially, the CSV file (NyDB.csv) is loaded into a file called ozzy.db, which serves as our database. The connection to the SQLite database file is established through `connection.cursor()`, allowing the program to execute SQL commands. The code opens the CSV file `static/NyDB.csv` using the `open()` function with UTF-8 encoding, allowing the code to handle special characters. The `csv.DictReader()` reads the file as a dictionary, where the column headers in the first row are used as keys for the values in the associated columns. The `delimiter=';'` specifies that the file uses semicolons to separate the columns.

The next piece of code ensures that all food items and information regarding macros and CO2 emissions are placed into a table created through `createtables.py`. The code iterates over each row in the `NyDB.csv` file, ensuring that the tables in `createtables.py` match the columns in the CSV file. It is essential that the columns in the CSV file match each value inserted into the table to ensure that data is correctly extracted, which allows us to comment out the code. Once `ID_Ra`, `Navn`, `CO2`, `Energi`, `Fedt`, `Kulhydrat`, and `Protein` are inserted into the `Food_product` table, the `connection.close()` saves the changes to `ozzy.db`, printing a confirmation message. Once this message is displayed, the code above is commented out with `"#"` to ensure stability and prevent accidental duplication of food items. Once the data is loaded into the SQL table `Food_Product`, the CSV file can be made inactive.

4.2.1 Construction of createtables.py

The database initialisation is essential in our web solution, as `ozzy.db` requires storage and management of user input, with the intention of performing a calculation that can inform the user of macros and CO2 emissions of the chosen food items. The following section will explain the `createtables.py` script.

```
db.execute("""
CREATE TABLE IF NOT EXISTS bruger (
    ID INTEGER PRIMARY KEY AUTOINCREMENT,
    Email TEXT NOT NULL,
    Password TEXT NOT NULL,
    Username TEXT NOT NULL
)
""")
```

The initial table named `bruger` stores user information once the user has submitted an email, password, and username through `registration.html`. `ID INTEGER PRIMARY KEY AUTOINCREMENT` allocates a unique ID once the user has been created, which is automatically generated. This unique ID is used later in the code and will be elaborated on in the `UserFoodLog` table. `TEXT NOT NULL` ensures that the backend has to receive an input from the user before the form can be submitted. The `CREATE TABLES IF NOT EXISTS` command ensures the table is created only once, preventing duplication even if the script is executed multiple times.

```
db.execute("""
CREATE TABLE IF NOT EXISTS Ulist (
    ID_U INTEGER PRIMARY KEY AUTOINCREMENT,
    Product_name TEXT NOT NULL,
    Grams INTEGER NOT NULL
)
""")
```

The `Ulist` table keeps track of the specific food items and quantities that the user has selected. `ID_U` serves as a unique identifier for each entry in the table, `Product_name` describes the food items, and `Grams` details the amount in grams of the chosen food.

```
db.execute("""
CREATE TABLE IF NOT EXISTS Food_product (
    ID_fp INTEGER PRIMARY KEY AUTOINCREMENT,
    ID_Ra TEXT NOT NULL,
    Navn TEXT NOT NULL,
    CO2 INTEGER,
    Energi INTEGER NOT NULL,

```

```

        Fedt INTEGER NOT NULL,
        Kulhydrat INTEGER NOT NULL,
        Protein INTEGER NOT NULL
    )
    """

```

The `Food_product` table is part of creating the database that contains all our food items. As described in the `ozzy.db` construction section of the report, a CSV file containing columns of food items and the associated information regarding CO2 emissions, fats, carbs, and protein is loaded into the table and then commented out in the `app.py` code. The `Food_product` table has to be created before loading the CSV file into the SQL database, making it crucial for creating the calculations from linear programming (LP). Essentially, the `Food_product` table serves as the core reference for calculating the nutrition values and CO2 emissions.

```

db.execute("""
CREATE TABLE IF NOT EXISTS UserFoodLog (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    product_name TEXT,
    grams INTEGER,
    rCO2 INTEGER,
    rEnergi INTEGER NOT NULL,
    rFedt INTEGER NOT NULL,
    rKulhydrat INTEGER NOT NULL,
    rProtein INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

```

The `UserFoodLog` table is a central part of the database as it connects the user's activity with the calculations of macros and CO2 emissions. `id INTEGER PRIMARY KEY AUTOINCREMENT` ensures each food entry is logged uniquely and can be referenced by the ID. `user_id` is a foreign key linking each food entry to a specific user in the `bruger` table. It allows tracking of food consumption by user. `product_name` links the logged food item to a specific entry in the `Food_product` table. This table is vital for calculations. Each time a user logs a food item, the input is stored in `UserFoodLog`, containing the amount of grams, allowing retrieval of necessary data from the `Food_product` table. Calculations for macros and CO2 emissions are performed by multiplying the nutritional values by the user's inputted quantity. Results are stored in `UserFoodLog` and displayed to the user through `search.html` and `frontpage.html`.

```

session['selected_foods'].append({
    'product_name': product_name,

```

```

    'grams': grams,
    'co2': round(co2_emissions, 2), # CO2 in kg
    'protein': round(protein, 2),
    'carbs': round(carbs, 2),
    'fat': round(fat, 2),
    'calories': round(energi, 2)
})

# Calculate total emissions and macros
total_co2 = sum(item['co2'] for item in session['selected_foods
    '])

```

The `sum()` function calculates the total CO2 emissions and macros by iterating through the list of selected foods in the session. The respective values are extracted and summed up, displaying the total amounts of protein, fats, carbs, calories, and CO2 emissions. In conclusion, the `createtables.py` script plays an essential part in initialising and managing the database `ozzy.db`. It sets up the necessary functions for creating the user, storing user data, storing food items, and processing user input. This enables accurate calculations of macros and CO2 emissions for any food item. Figure 4.2 shows how the database tables interact with each other.

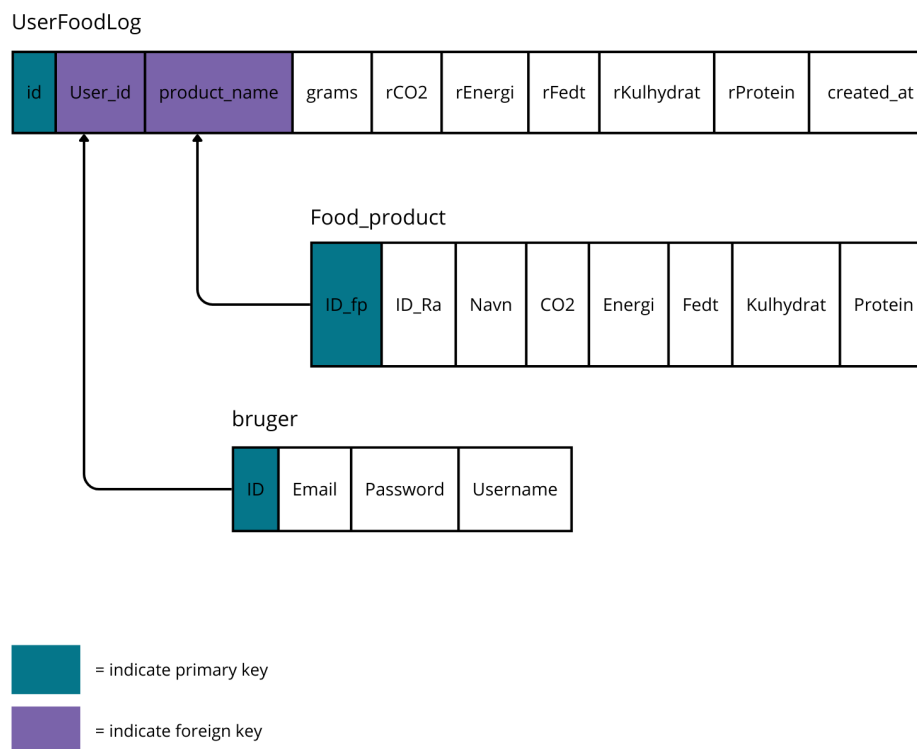


Figure 4.2: schema of how tables in database are connected.

4.3 Requirements

The following section describes each of the requirements for the construction of our web solution. The requirements are the ones from figure 3.1. This section will explore how we have fulfilled the requirements with the logic behind developing the code. Code snippets from our web solution will also be included.

4.3.1 Requirement 1 - User Login

The first requirement concerns a personal user login section, which is something most applications have. This ensures the product becomes personal for the specific user, both their inputs about food and their goals. This requirement originates from the flowchart located and developed during the abstraction phase 4.1. For R1 we have created two html files and three app.routes in our app.py file. To implement the register functionality, our HTML file includes the following form:

```
<h1>Register</h1>
<form action="/register" method="post">
  <label for="Email">Email:</label>
  <input autocomplete="off" id="Email" name="Email" placeholder="
    Enter_your_email" type="Email" required>
  <label for="Username">Username:</label>
  <input autocomplete="off" id="Email" name="Username"
    placeholder="Enter_your_username" type="Username" required>
  <label for="Password">Password:</label>
  <input autocomplete="off" id="Password" name="Password"
    placeholder="Enter_your_password" type="Password" required>
  <label for="password\_repeat">Repeat Password:</label>
  <input autocomplete="off" id="password\_repeat" name="password\_
    _repeat" placeholder="Repeat_your_password" type="Password"
    required>
  <button type="submit">Register</button>
</form>
```

The `<form>` element specifies that data should be sent to the server via a POST request to the `/register` route using `method="post"`. The required attribute ensures that all fields must be filled before submission. The user is prompted to input their email, username, and password, which the server receives for processing. In the backend, Python and Flask allow user registration, with SQL storing their data in a database table. (OpenCourseWare - Harvard 2024). The purpose of `app.route("/register")` is to receive the form data, validate it, store it in the database, and redirect users to the login page upon success. Here's the corresponding Python code:

```
@app.route("/register", methods=["POST"])

def register():
    Email = request.form.get("Email")
    Password = request.form.get("Password")
    Username = request.form.get("Username")
    return render_template("failure.html", message=str(e))
```

The `app.route` works by processing the form data sent via POST. The `request.form.get()` retrieves the values of form fields with names matching those in the POST request. Flask's request object handles HTTP requests, and its `form` property allows access to form data (Pallets Projects 2024a). The `get()` method retrieves the value associated with the specified key.

For example, when the user submits the form, their email, username, and password are extracted from the request and can be validated and stored in the database. This combination of HTML and Flask ensures a smooth registration process for users. Similarly to the `index.html` file, we have included a measurement to ensure we receive all of the user's inputs in the `app.py` file. If the user has only filled out one or none of the input fields, this piece of code will make them aware of the mistake. We have done so by using a return statement to show the user our HTML page called "failure", which shows the user an error message.

```
if not Email or not Password:

    return render_template("failure.html", message="Both_fields_
        are_required.")

try:
    db.execute("INSERT INTO bruger (Email, Password, Username)
        VALUES (?, ?, ?)", Email, Password, Username)
    return redirect("/login")
except Exception as e:
    return render_template("failure.html", message=str(e))
```

If all fields are successfully completed by the user, their inputs about their email, password and username will be inserted into a table called `bruger` in our database. This is done with `db.execute`, which is a SQL statement that executes a SQL query (OpenCourseWare - Harvard 2020). Lastly, we have included error handling, in case something goes wrong with inserting into the database. The user will see the "failure" HTML page with an error message, if that happens. The next step in meeting the R1 requirement is to make a login page for the user, and our form in the `login.html` is the following:

```
<form action="/login" method="post">
```

```

<h1>Login</h1>

<input autocomplete="off" autofocus name="Email" placeholder="
    Email" type="email" required>

<input autocomplete="off" name="Password" placeholder="Password
    " type="password" required>

<!-- Error Message (conditionally displayed) -->
\{% if error \%}
<p style="color:_red;">\{{ error \}}</p>
\{% endif \%}
<button type="submit">Log in</button>
<label>

```

The login form is designed to collect the user's email and password and send this data to the `/login` route on the server via a POST request. It includes fields for the email and password, each equipped with attributes like `required` to ensure all necessary inputs are provided. The `autofocus` attribute in the email field automatically places the cursor there when the form loads, enhancing usability. Additionally, if the server detects an issue, such as invalid input, an error message appears. Once the form is submitted, the server processes the inputs and verifies the credentials against the database to determine if the login is successful. This setup ensures a straightforward and interactive user experience.

The purpose of `app.route/login` is to check whether the inputs from the user in `email` and `password` are already in the `bruger` table and inform the user if the inputs are invalid. This code also makes a `user_id` for the user, which is not something the user will see, but something we will use to manage each user's session. The `app.route/login` works by retrieving the form data from the user by asking about email and password. We have made the error messages in case of missing input similarly to those in `app.route/register`.

```

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        Email = request.form.get("Email")
        Password = request.form.get("Password")
        if not Email or not Password:
            return render_template("login.html", error="Email_and_Password
                _are_required")

```

We ensure the user exists by comparing the email input on the login page with the entries in the table `bruger` in our database. This is done using an SQL query, as shown below and if the user

is in the `burger` table, we proceed to check the user's input in password as well. Thereafter we create a `user_id` for the user session, which stores the user's unique identifier. This id will be used later on in the search function.

```

user = db.execute("SELECT_*_FROM_bruger_WHERE_Email=_?", Email
)

if not user:
    return render\template("login.html", error="Invalid_email_or_
        password")

if user[0]["Password"] == Password:
    print("Login_successful")
    session["user\_id"] = user[0]["ID"]
    session["Email"] = Email
    session["Username"] = user[0]["Username"]

```

If the user has entered the correct email and password and is already registered, they will be redirected to the frontpage. If not, the user will receive an error message.

```

return redirect("/")
else:
    return render\template("login.html", error="Invalid_email_or_
        password")
return render\template("login.html")

```

4.3.2 Requirement 2 - Search food item & calculation of macros & CO2

The second requirement details how the user needs to be able to select food items from our database and input amount of grams, resulting in an instant calculation that will inform the user about the amount of calories, carbs, fats, protein and finally the CO2 emission of the food item chosen. Requirement 2 is an important part of creating our web solution, derived from the flowchart 4.1. To initialise this specific function, the first step is to ensure that the user is prompted for inputs regarding choosing a food item and amount of grams on the `search.html` page. The `search.html` file serves as the interface, with it being dynamically updated based on whether the search result is available.

```

{% if results %}
<h3>Search Results:</h3>
<form method="POST" action="{{_url_for('search')_}}">
    label for="Navn">Product Name:</label>

```

```

<select id="Navn" name="Navn" required>
  <option value="" disabled selected>Select a product</
  option>
  {% for product in results %}
    <option value="{{_product[0]_}}">{{ product[0] }}</
    option>
  {% endfor %}
</select>

```

If the user types in a food item that matches with our database, the if results will activate the following section of code. The user will be presented with a dropdown menu listing the available products matching the query, with each food item in the dropdown corresponding to the product name retrieved from the search result. This is done through the for loop, where each product in the results list is iterated over, and the `<option>` element is created for each product. The value attribute of the `<option>` contains the product name (`product[0]`), with it also being displayed in the dropdown menu, ensuring that the user can choose from only the matching products according to the letters that are inputted into search.

```

<label for="grams">Grams:</label>
<input type="number" id="grams" name="grams" required>

```

Once the user has selected a product, the user will be prompted to input the quantity of the product in grams. This occurs through the Grams Input, where a numeric input field prompts the user to specify the amount in grams, with the `required` attribute ensuring that the user can not leave the field empty, ensuring that all necessary data is collected. Once the user has selected a product, the user will be prompted to input the quantity of the product in grams. This occurs through the Grams Input, where a numeric input field prompts the user to specify the amount in grams, with the ‘required’ attribute ensuring that the user cannot leave the field empty, ensuring that all necessary data is collected.

```

<button type="submit">Submit</button>
</form>
{% else %}
  {% if request.args.get('search') %}
    <p>No results found. Please try again.</p>
  {% endif %}
{% endif %}

```

Once the user clicks submit, the input will be checked with `ozzy.db` to ensure that the input corresponds with the database of food items. If none of the food items match the user’s search query, the `% else %` block is activated, displaying "No results found."

```

if request.method == "POST" and 'Navn' in request.form:

```

```
product_name = request.form.get('Navn')
grams = int(request.form.get('grams'))
```

If the query matches the database `ozzy.db`, it is passed through the POST request. This is a condition that handles user inputs the server needs to process. "POST" and 'Navn' ensure that the form cannot be submitted without the user choosing a food item, retrieved using `request.form.get('Navn')`. The same process occurs for the quantity in grams, retrieved through:

```
int(request.form.get('grams'))
```

This ensures the retrieval of a numerical value.

```
query = """
    SELECT ID_Ra, Navn, CO2, Energi, Fedt, Kulhydrat, Protein
    FROM Food_product
    WHERE Navn LIKE ?
    """
cursor.execute(query, ('%' + product_name + '%',))
result = cursor.fetchone()
```

The code executes a query to search `ozzy.db`, the database with all the food items, with the user's input regarding the food item and amount in grams. The SQL query selects specific columns (ID_Ra, Navn, CO2, Energi, Fedt, Kulhydrat, and Protein) from the `Food_Product` table. The `WHERE Navn LIKE ?` condition allows for partial matching of the user's input, e.g., "bee" returns items containing "bee". The query result is fetched using `result = cursor.fetchone()`.

```
product_id = result[0]
product_name = result[1]
co2_per_kg = float(result[2]) # CO2 (per kg)
energi_per_100g = float(result[3]) # Energy (kcal per 100 g)
fat_per_100g = float(result[4]) # Fat (g per 100 g)
carbs_per_100g = float(result[5]) # Carbs (g per 100 g)
protein_per_100g = float(result[6]) # Protein (g per 100 g)
```

If the user input matches a product, details are extracted from `ozzy.db` to calculate CO2 emissions and macros. Each value is converted to `float` for precise calculations.

```
fat = fat_per_100g * grams / 100
carbs = carbs_per_100g * grams / 100
protein = protein_per_100g * grams / 100
energi = energi_per_100g * grams / 1000
# Calculate CO2 in kilograms (since it's stored per kilogram in
    the database)
```

```
co2_emissions = co2_per_kg * grams / 1000
```

The macros and CO2 emissions are calculated based on the quantity of the specified product. Each value is scaled appropriately to reflect the user's input. For example, `protein_per_100g` is multiplied by the user's input and divided by 100.

```
calories = (protein * 4) + (carbs * 4) + (fat * 9)
```

The database containing all food items located in the Food product table does not calculate the amount of calories of the selected items. Therefore, a formula calculating the amount of calories is necessary for the user to track their nutrition. Proteins and carbohydrates each contribute 4 calories per gram, with fats contributing 9 calories per gram.

```
co2_emissions = co2_per_kg * grams / 1000
```

The CO2 emission per food item is listed in kilograms in our database, emphasising how a formula is needed in order to convert it to grams, which occurs when fetching the result from the database and dividing by 1000.

4.3.3 Construction of remove button

Allowing the user to choose multiple food items, and subsequently calculating the macros and CO2 emissions highlights the necessity of allowing users to remove any or all of the chosen food items. The remove button logic can be seen in figure 4.1 on the search page section.

```
if request.method == "POST" and 'remove_food' in request.form:
```

The code checks if the HTTP request is a POST request, specifically if the form contains the key linked to `remove_food`. The `remove_food` key corresponds to the value submitted when the user hits the remove button.

```
food_index = int(request.form.get('remove_food'))
```

The `remove_food` field holds the index of the specific food item that the user wants to remove. This is important, as we want the user to be able to remove a specific food item from the list. The value is retrieved through `request.form.get()` and converted to an integer using `int()`, ensuring that the index is usable for operating a list of different food items.

```
selected_foods = session.get('selected_foods', [])
```

Once the user chooses multiple food items and inputs the quantity, 'session' is used to temporarily store user-specific data. The `get()` method retrieves the list of food items from the session, which is stored in `[]` if no items exist.

```
if 0 <= food_index < len(selected_foods):
    selected_foods.pop(food_index) # Remove the selected food
    item
```

```

session['selected_foods'] = selected_foods # Update the
    session with the modified list
conn.commit() # Commit any changes if needed
return redirect("/search") # Redirect to update the page
    after removal

```

The code initially checks if there are any food items that could be removed through `selected_foods`, preventing runtime errors. If there are food items, the `pop()` method removes the item specified by `food_index` from `selected_foods`. The session is then updated with the modified list, ensuring the specific food item is removed while still displaying the remaining items. Finally, the user is redirected to `/search` to update the page and reflect the changes.

4.3.4 Requirement 3 - Display macros & CO2 emissions

The third requirement of our project focuses on visualising the users' total of macros, kcal and CO2 emissions. This feature is inspired by insights gained from WOZ, which demonstrated the value of presenting user data in a visual format. The aim was to provide the users with an intuitive, aesthetically pleasing, and easy to understand representation of their nutrition and environmental impact. Visualisation simplifies complex data, making it easier to track and adjust habits for the user. The flowchart (figure4.1) shows the structure of the flow, providing a clear overview of the system's processes and interaction. Drawing inspiration from apps such as Floop and MyFitnessPal, which display user data on their frontpages, we decided to integrate the visualisations on the frontpage. This feature includes two types of visualisations:

1. a pie chart showing the breakdown of nutritional values
2. a scatter chart showing the balance between calories and co2 emissions against an average baseline.

This requirement involves two main files: an HTML file for the frontpage and a python file to handle the backend logic and data processing. The HTML file, `Frontpage.html`, handles the frontend interface, while the python file, `app.py`, manages the backend logic. Only a section of the `app.py` file contains the relevant code for this requirement, this section is the `@app.route()`. The `app.route` functions performs by first looking at user authentication, here the user is logged in by checking if the `user_id` is in session, if not the user is redirected to `/login`. Then we reuse the variables from the `/search` to retrieve the data from the user's selected foods (explained in section [subsection 4.3.2](#)). This is done through `selected_foods = session.get('selected_foods', [])`, All the different values are summarised in variables `total_protein = sum(item['protein'] for item in selected_foods)` This data is stored in the `macro_data` variable. This is a 2d array, for the macronutrition data, which will be used to create the pie chart.

```

macros_data = [
    ['Macronutrient', 'Amount'],

```

```

    ['Protein', total_protein],
    ['Carbs', total_carbs],
    ['Fat', total_fat]
  ]

```

Each entry in the array includes one of the macronutrients and their associated value. To generate the charts, we use Google charts, which provides ready-to-use charting tools. For this project the scatter chart and the pie chart are used. Three elements are required to use Google Charts: a `<div>` element with a unique id, a link to load in the charts and to load in the Graph API, and add the function to run it. The following code snippet is cut outs showcasing the aforementioned elements.

```

<script type="text/javascript" src="https://www.gstatic.com/charts/
  loader.js"></script>

google.charts.load('current', { packages: ['corechart', 'line', '
  scatter'] });

google.charts.setOnLoadCallback(drawMacrosChart);
google.charts.setOnLoadCallback(drawScatterChart);

function drawMacrosChart() {
}

<div id="scatterChart" class="chart-container" style="
  height:_500px;"></div>

```

To create the functions the pie chart, the `macrosData` was imported from the python file through the Google Charts visualisation. The basic information's are added, like title, size and colours.

```

function drawMacrosChart() {
  const macrosData = google.visualization.
    arrayToDataTable({{ macros_data | safe }});
  const options = {
    title: 'Macronutrient Breakdown',
    pieHole: 0.4,
    colors: ['#FF6384', '#36A2EB', '#FFCE56']
  };
  const chart = new google.visualization.PieChart(
    document.getElementById('macrosChart'));
  chart.draw(macrosData, options);
}

```

The second function is created through a scatter chart. This chart provides a simple comparison in which calories and CO2 emissions are used as datapoints, compared to an average person.

```
function drawScatterChart() {
  const combinedData = new google.visualization.DataTable();
  combinedData.addColumn('number', 'Calories (kcal)');
  combinedData.addColumn('number', 'CO2 Emissions (kg CO2)');
  combinedData.addColumn({ type: 'string', role: 'style' });
  combinedData.addColumn({ type: 'string', role: 'tooltip' });
  ;

  combinedData.addRow([2000, 5, 'point {color: #DDC5A9; size: 6}', 'Average Person']);
  combinedData.addRow([{{ total_calories }}, {{ total_co2 }},
    'point {color: #6FA96F; size: 8}', 'Your Data']);

  const options = {
    title: 'Calories vs CO2 Emissions',
    hAxis: { title: 'Calories (kcal)', minValue: 0,
      maxValue: 3000 },
    vAxis: { title: 'CO2 Emissions (kg CO2)', minValue: 0,
      maxValue: 10 },
    legend: 'none',
    pointSize: 8
  };

  const chart = new google.visualization.ScatterChart(
    document.getElementById('scatterChart'));
  chart.draw(combinedData, options);
}
```

The first section of the code involves the preparation of the data, this includes adding the columns, with the specifications of the column like numeric values, and the style for the datapoint. The datapoints are created by adding rows, here one row is hardcoded values, representing the average person. As many parameters are involved in specifying an average person, for the sake of this web solution, the calorie value is set to 2000 kcal, and emission is set to 7.1 kg ([Eaternity, n.d.](#)). The second point is created with the dynamic values retrieved from the Python file, thus `total_calories` , `total_co2` . These elements will then go into a chart with decided behaviour and appearance. In the following code snippet:

```
const options = {
```

```
title: 'Calories vs CO2 Emissions',  
hAxis: { title: 'Calories (kcal)', minValue: 0,  
        maxValue: 3000 },  
vAxis: { title: 'CO2 Emissions (kg CO2)', minValue:  
        0, maxValue: 10 },  
legend: 'none',  
pointSize: 8  
};
```

The id's for the charts `<div id="scatterChart">` and `<div id="macrosChart">` are then used in the `<style>` portion of the HTML to integrate them. The user can access this feature by logging in, which directs them to the `@app.route("/")`. The charts are only displayed when food has added to the selected foods on the search page. If food items are selected, charts will display the user's intake of their macros separated into protein, carbs, and fats, along with their calories and CO2 emissions, based on the selected foods. The manner in which we chart is inspired by linear programming except the factors involved in displaying the results are changed to involve only macronutrients (fats, protein and carbohydrates), calories and CO2 emission since that is the focus of our web solution.

If the user adds more food, the charts will update as well. This is be beneficial to the user as the charts simplifies the complex nutritional and environmental data. The users can track their dietary balance, by identifying potential over or under consumption of the macro nutrition. They will also be able to view the environmental impact of an average person. The implementation of Google Charts creates a simple and responsive user experience. This visualisation meets the requirement, by giving a visual overview of the personal nutrition and environmental impact.

4.3.5 Requirement 4 - Display Results

The fourth requirement describes how our web solution should display the total of macros and CO2 emissions to the user. The system is designed to provide users with a deeper understanding of how their food choices influence the environment. By allowing users to select various food items, calculate the associated CO2 emissions, and compare these emissions to the average emissions of a typical Danish meal, the platform aspires to raise awareness about the environmental impact of dietary habits.

At the core of the system is its ability to collect, calculate, and compare data on CO2 emissions. It begins by gathering information from `ozzy.db` on the emissions produced by various food items. This data serves as the foundation for the platform's calculations and insights. When users select specific food items, the system saves the corresponding emissions data in a session variable called `selected_foods`. This feature allows the system to dynamically track user choices and update calculations in real time, creating a seamless and interactive experience. The system aggregates

the total CO2 emissions by iterating through the list of selected food items and summing up their respective CO2 values. For example, a concise Python snippet illustrates this process:

```
Calculate total CO2 emissions from selected items
total_co2 = sum(item['co2'] for item in selected_foods)
```

The emissions data for each food item originates from a trusted database, ensuring that the results presented to users are accurate and dependable. This transparency and precision are critical for building trust and encouraging meaningful user engagement. To give users a point of reference, the system incorporates an average emissions value for a typical Danish meal. This benchmark is set by us at 7.1 kg of CO2 emissions and serves as a comparative baseline. While the specific value is made by us, it provides a useful standard for evaluating the environmental impact of individual meals. In the code, this benchmark is defined as follows:

```
Define the benchmark for average meal emissions
average_co2 = 7.1 # Average CO2 emissions per meal in kg
```

The system then compares the user's meal emissions to this benchmark and offers feedback tailored to the results. Using conditional statements, the platform evaluates whether the user's meal emissions are higher, lower, or equal to the average. Based on this evaluation, it generates a message to inform the user of their meal's relative environmental impact:

```
# Determine the CO2 comparison message and color
if total_co2 > average_co2:
    co2_comparison = f"Your meal emits {total_co2 - average_co2:.2f} kg more CO2 than the average daily intake per day ({average_co2:.2f} kg)."
    co2_color = "red"
elif total_co2 < average_co2:
    co2_comparison = f"Your meal emits {average_co2 - total_co2:.2f} kg less CO2 than the average daily intake per day ({average_co2:.2f} kg)."
    co2_color = "green"
else:
    co2_comparison = "Your meal emits the same amount of CO2 as the average intake per day."
    co2_color = "yellow"
```

To give the user a better understanding, the system employs colour-coded feedback. Red means a higher environmental impact, green means a lower impact, and yellow is an equal impact compared to the average. This visual element simplifies complex data and makes it more accessible. Furthermore each of these colours have a corresponding emoji that displays, either a happy face

for green, indifference for yellow and unhappy for red. The Nutritional Overview Section is the primary interface where users can review their meal's CO₂ emissions and see how it compares to the average. This section is made in a simple manner and with clarity in mind to ensure user accessibility, constructed with two key components. The layout uses `display: flex;` to arrange the content in a way that works well on different screen sizes, from desktops to smartphones. This part uses a table to show the total CO₂ emissions in a straightforward way. The table has two columns: one for the label `Total CO2:` and another for the value `total_co2 kg`.

The system's design prioritises accessibility, ensuring that all users can understand the given insights. Clear text, high-contrast colours, and responsive layouts cater to user needs. The use of visual feedback, such as colour coding and corresponding emojis further enhances comprehension by allowing users to quickly understand the implications of their meal choices, in a different manner than just numbers. Additionally, the system's simplicity and clarity play a role in user engagement. By presenting information in an intuitive format, it lowers the barrier to understanding and encourages users to interact with the platform more frequently. These design principles aim to make environmental awareness both approachable and action more possible.

This system serves as an educational and motivational tool, helping users understand the environmental impact of their dietary choices. By combining accurate CO₂ calculations with user friendly design elements, it encourages more sustainable eating habits. Through clear comparisons, visual feedback, and accessible interfaces, the platform effectively bridges the gap between complex environmental data and everyday decision-making.

4.3.6 Requirement 5 - Transparency

The fifth requirement focuses on ensuring transparency between the web solution and the user. To ensure transparency, we have implemented an "information" page located through `information.html`, following the same structure as `search.html` and `frontpage.html`, with all pages accessible on through the navigation bar. The flowchart 4.1 highlights the importance of the information page as the foundation for building trust and informing users about our data and calculations, while also serving as the platform for transparency. The transparency page explains how the food data originates from "Klimadatabasen" (CONCITO 2024a). The database also described in the 4.1 database section, measures the emissions of food products in kilograms of CO₂.

```
"food_product*_your_input_in_grams_/_1000, "
```

The calculation that occurs through our backend is explained on `information.html` in the transparency section to provide clarity for the user. Ensuring instant access to `information.html` on all pages of the web solution emphasises the commitment in showing transparency to the user. The construction section describes how we managed to create a web solution that provides a tool to help young Danes track and improve their nutritional intake while reducing their CO₂ emissions.

The flowchart 4.1 served as the starting point for creating the web solution. It provided a visual overview of the systems structure, including the relationship between the frontend, backend and database. The flowchart outlined the user's journey through the web solution, from the starting point of creating an account, inputting food data to receiving calculations and visualisations. By mapping out key functionalities and ensuring logical flow, the flowchart acted as the blueprint that guided the construction process, ensuring consistency alignment with our requirements.

4.4 Design choices

As mentioned in the introduction, there is a lot of misinformation when it comes to both nutrition and CO2 emissions. This has reflected on our design process and choices. Our aim regarding the layout has been to create a simple and intuitive tool for the users. The navigation bar, as shown in figure 4.3, makes it easy for the users to navigate the web solution. Figure 4.3 shows the top of the frontpage and figure 4.4 shows the bottom half. These two figures gives the user information about how their macros are divided across their meals and how much CO2 they have emitted through their daily food intake. It also shows the user how much their CO2 emissions is compared to the average emissions.

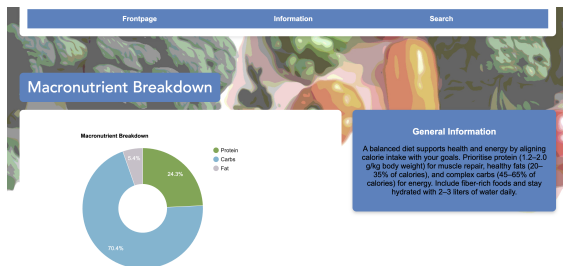


Figure 4.3: Screenshot of Macronutrient Breakdown on frontpage.

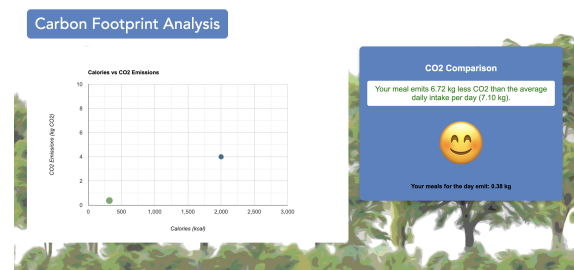


Figure 4.4: Screenshot of Carbon Footprint Analysis on frontpage.

Similarly to the frontpage, the search page embraces the a simple layout and it contains the same navigation bar. This page allows the users to search for a food item, and then adds the food item to a table on the page. From there the user can see their total nutritional overview and a CO2 comparison element, which can be seen in figure 4.5. The colour choices were made to have a simple appearance, allowing for a clean and professional look that emphasizes clarity and readability. The soft tones and contrast between elements ensure the design remains visually appealing and user-friendly.

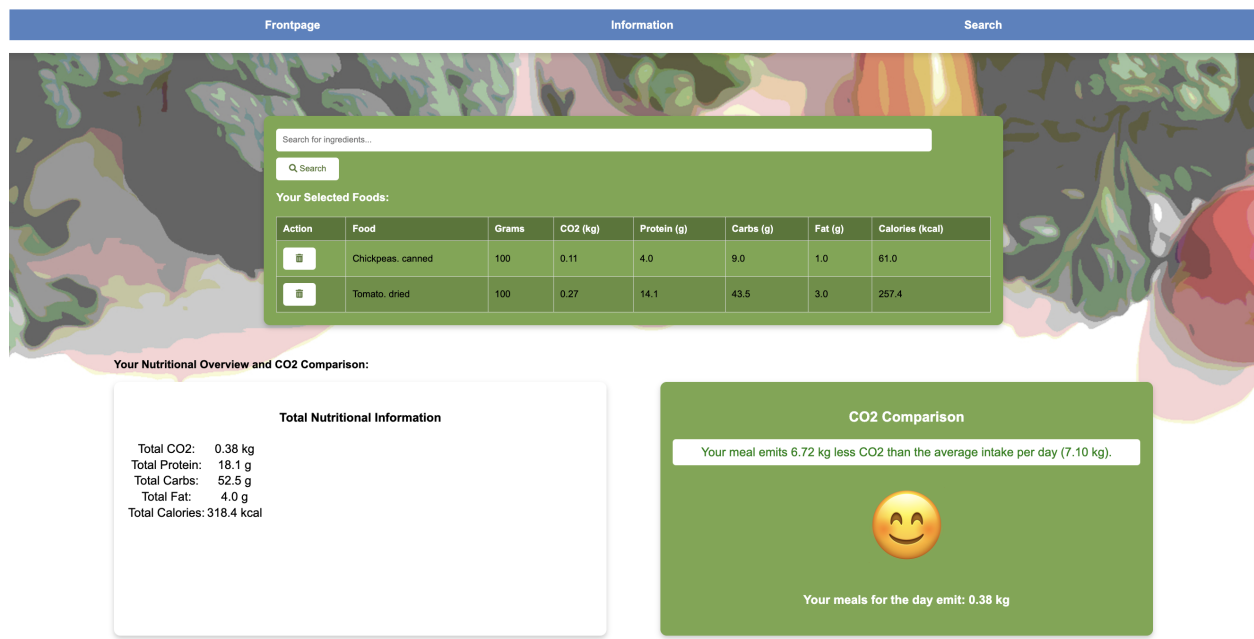


Figure 4.5: Screenshot of search page.

5 Discussion

Our discussion will reflect on how well our project requirements were fulfilled while we will explore the different challenges and limitations regarding our data and construction. We sought to create a tool for tracking nutritional intake alongside CO₂ emissions from food. Our entire project is built on data from the “Klimadatabasen” and the survey from CONCITO about young Danes and their willingness to change their nutrition in order to be more climate friendly. In the first section of our discussion, we discuss the benefits and possible limitations of the data and survey used in our project, providing insight into how they contribute to the understanding of the problem.

5.1 Klimadatabasen

The data which our project is based on is from “Klimadatabasen”. The data in this has been published by CONCITO and it is financed and done by 2.-0 LCA Consultants (CONCITO 2024b). Both of these companies seek to contribute to sustainable development and convey climate friendly solutions for citizens, companies and politicians (CONCITO 2024d). “Klimadatabasen” was created to meet the demand from consumers who are increasingly seeking information about the CO₂ emissions of the products they purchase. To meet this demand, “Klimadatabasen” offers free access to information about CO₂ emissions for over 500 common food items. The database provides detailed information in kilograms of CO₂ equivalents (CO₂e) per kilogram of product, capturing the impact of CO₂, methane, nitrous oxide, and land-use changes. It also evaluates food purchases for businesses and households, ranks food types for simplified climate labelling, and educates the public about the climate impact of food choices (CONCITO 2024b).

However, the database has certain limitations. The average CO₂ emissions values do not account for the wide variations within individual product categories. Furthermore, it does not provide a comprehensive sustainability assessment, as it excludes other sustainability factors. These factors include but are not limited to animal welfare, biodiversity, food waste and general health. To draw comprehensive conclusions on sustainable food systems, it is essential to consider all of these factors (CONCITO 2024c, p. 42). “Klimadatabasen” is not intended as a dietary guide, since minimising the CO₂ emissions of food alone does not ensure a nutritionally balanced diet. The data provided is dynamic and meant solely for general orientation and education, rather than for regulatory purposes (CONCITO 2024b). By choosing “Klimadatabasen” we have not only used the latest and best data for our specific problem statement, but it also provides a “frame” for our

web solution. By using “Klimadatabasen” we do not have all food types and variations available, but trying to collect a dataset like that would be complex. “Klimadatabasen” is a reliable tool for us to base the project on as long as we make sure we provide transparency about the data and its limitations to the user.

5.2 CONCITO Survey

The survey from CONCITO concluded that half of Danes between the age of 18-29 want to change their diet but lack adequate knowledge of how to do it. Therefore, our project sought to create a web solution that ultimately provides a tool to track both their nutritional intake and their food-related CO2 emissions. The data from CONCITO's survey, “Unges madvaner under forandring,” published in February 2021, gives a concise insight into the relationship between nutritional habits and climate awareness among young people in Denmark (Clara Laurine Berg-Jensen 2021). The report highlights an increasing concern among young Danes, specifically in the 18-29 age group. This is critical, as it establishes a strong foundation for the notion that young people are aware of climate issues, but this awareness does not necessarily translate into climate-friendly dietary choices. The survey consisted of a set of questions that provided a deeper understanding of young people's dietary choices, including questions regarding their consumption of meat and attitudes towards eating less meat. While the survey provides us with valuable insights, there might be limitations to consider.

The survey by CONCITO, while representative of a large sample due to the 3,000 participants selected based on gender, age, region, and educational level, may not fully capture the diversity of young Danes' experiences across Denmark. The data could be influenced by regional differences, where urban areas might have different nutritional habits compared to rural areas. Additionally, while the data provides a broad snapshot of young Danes' attitudes towards nutrition, it may not account for specific subgroups within the youth population, such as those with varying levels of income or those living in different socioeconomic conditions, which could affect their food choices and ability to follow a sustainable diet. Additionally, the survey questions related to meat consumption and climate-friendly eating rely on self-reported data, which might create the risk of participants being biased and potentially exaggerating their environmentally conscious behaviours to align with social expectations or what they believe is considered acceptable.

Despite these possible limitations, the survey made by CONCITO provides valuable insight into the attitudes and behaviours of young people in Denmark regarding climate-friendly food choices, serving as a useful starting point for improving knowledge regarding climate-friendly nutrition. The survey stresses the willingness for young Danes to change their dietary patterns, but also the difficulty in doing so. It served as the starting point for creating our problem statement and has since paved the way in which we have constructed our entire web solution.

5.3 Dietary recommendations

Due to the complexity of creating dietary recommendations as highlighted by Nestle (section 2.2), which we explored in the related works section, our project solely focused on creating a web solution showing the macros and CO₂ emissions. Offering specific guidance on sustainable or nutritious diets, offers a significant challenge due to how the food industry increasingly focuses on marketing their products as sustainable and nutritious, without necessarily living up to the standards of what could be deemed as such. Categorising any food item as inherently sustainable oversimplifies the complexities of food systems. However that is not to say that all data concerning food production and the nutritional profile of foods are unusable, just that awareness and transparency is necessary. We have for used food data from “KlimaDatenbanken” (CONCITO 2024a) and we have calculated the numbers for our visualisations using LP for the scatter chart, percentages for the pie chart and simple arithmetic for the users average CO₂ emissions. This made it so we focus on the users’ input about their daily food intake, as well as limiting the categories we display for user guidance to CO₂, calories and macronutrients. Ultimately giving the users of our web solution a chance to make well informed choices and balance their nutrition and emissions.

The EAT-Lancet Commission’s report (section 2.1) on healthy diets from sustainable food systems provides a critical insight into the difficulties of balancing nutritional intake and environmental sustainability. The report introduces the concept of a “reference diet” that integrates nutritional needs and CO₂ emissions from food production, recommending a focus on plant-based foods and reduction in animal-based products such as poultry and red meat. However, while the reference diet provides an important framework for balancing nutrition and sustainability, we do not adopt or attempt to create a reference diet. Our approach centers around offering a practical tool that visualises the CO₂ emissions of food choices. The challenge of balancing nutritional intake with sustainability remains at the core of our project, but rather than following and recommending a predetermined “reference diet”, we focused on informing the users of the CO₂ emissions of food intake. Our approach acknowledges that sustainability and nutrition is not a one-size-fits-all solution, but it allows users to make decisions that are both nutritionally sound and environmentally conscious.

5.4 Requirements

Our requirements for the project stem from the use of CT, the WOZ approach, and our related works section. These elements serve as the starting point for identifying the necessary features to be implemented in our web solution. The first five requirements were implemented to establish a baseline functional web solution, while the last three were prioritised for later development. The following section will discuss the requirements, what could have been done differently, and the

necessity of each requirement. Each requirement was informed by ongoing testing of the web solution during the development phase, with our flowchart serving as the outcome of each stage in creating the solution. Consequently, the discussion points regarding the requirements arise from a trial-and-error process, where we acted as the test group.

R1 describes how the system must enable users to have a personal account. From our related works section, it was clear that the apps we were inspired by had personal user accounts. We believed that it was necessary to enable users to have their own accounts, as the nutritional intake has to be connected and stored to a specific user. The system functions as a prototype and is currently stored on a local server, however, if the program has to be launched to the public, personal logging will be necessary. The program is currently placed on a local server where personal logging might not be necessary as it is connected to the local host and no other can access the current session. However, we chose to implement the personal login to simulate how the web solution would work if launched to the public. R1 was not a necessity for the basic functionality of the program, but it was nevertheless important for the web solution we wanted to create.

R2 describes how the system should prompt the user for a food item, store the quantity in grams and ultimately calculate the macros and CO₂ emission. For this requirement we were inspired by the apps from related works and the WOZ approach. However, in our program we only use the unit of grams. This works in the program, but it could be elaborated further so the program could also take other unit inputs from the user. These could be units such as tablespoon, millilitres and such. It could be further improved by using the same approach as the one in Floop, which gives the user an estimate of weight of common household foods. This could improve the user experience, by making it more accessible and not dependent on having to weigh everything the user consumes.

Our database of food items stems from “Klimadatabasen”, where all items were inputted from the onset. The database also provided the CO₂ emissions, which is necessary information for us to inform users of how much CO₂ any given food item contains. While “Klimadatabasen” is useful, as we did not have to build a database with all the information ourselves, it also restricts our program to only accept fixed inputs. A way to counter this potential problem is to allow the user input their own data about the food they eat. However this method is not foolproof. Myfitnesspal encounters issues with this regarding user diligence for accurate logging. Self-reported data can be prone to inaccuracies, whether due to underreporting, overestimation, or errors in portion sizes (Gioia et al. 2023). The only way to counter this is by expanding the users knowledge on proper logging. This issue would be even greater in our program as the users not only need to log data about the nutritional content of the food items, but also the amount of CO₂ it emits, which is rarely stated on the packaging. Furthermore, the survey from CONCITO highlights the inadequate knowledge regarding emissions in our food.

The “Remove” button was not initially a part of the requirement, however as we constructed the search function and tested it, we realised that sometimes errors occur with the logging of input. Therefore we found it essential to add a “remove” button, so it is easy for the user to fix mistakes.

We argue that the button is functionally necessary, as it improves the overall user experience and counters the fact that our program does not restart by the end of the day. Ideally, the program should differentiate between different days and automatically restart, so that the program is usable for more than merely a single day.

R3 and R4 describes how users should be shown a comparison to the average Danish person in regards to how much CO₂ the chosen food item emits, which was inspired from the WOZ approach. The feature was successfully implemented with the average set at 7.1 kilo CO₂ per day. However, the amount is entirely fictional. An average regarding CO₂ emissions cannot be determined with absolute certainty due to the complexities of food production systems and individual diets. The amount is not universally representative due to vast differences in regions, diets and production. However, we believed that the function was imperative in order to convey the amount of CO₂ in a tangible and relatable way. Floop demonstrates that it is possible to provide a plausible estimate by using extensive datasets and algorithms to calculate CO₂ emissions, but it necessitates extensive research and access to data much more comprehensive. Ideally our web solution would work similarly to Floop, which visualises the data through more concrete comparisons such as the following example “Boiling 1 litre of water 180 times in an average kettle or around 70% lower than the average persons CO₂e from food”. Nevertheless, we believed that emojis provides an immediate and instant indication of CO₂ emissions, ensuring that our target group can quickly understand whether their choices are good, average or poor.

The intention of the project is to highlight the environmental impact of nutritional intake and even an approximate number will highlight the large difference in emissions of CO₂ for different food items. Therefore, we decided to implement a comparison system that displays the total CO₂ emissions of the selected food items with a benchmark value of 7.1kg CO₂ per day. The comparison is visualised through three different emojis representing either lower, equal to or higher emissions than the set benchmark. The emojis makes the result of the comparison more understandable and lets the user understand through visual feedback. Building on the goal of providing relatable comparisons, the scatter chart serves as a visual representation of data, offering an overview of a users’ calories and the subsequent macros and CO₂ emissions. Currently, the calories are fixed at an average of 2000 per day - initially, the idea was to allow the user to input their own daily calorie goal depending on whether the user wanted to maintain, increase or lower their weight. However, we did not end up implementing this requirement about user inputted goals due to the project being an iterative process and the joint vision changing underway and therefore this graph was changed to an average person per day. Furthermore, it was not a necessary requirement for our web solution to work, and was therefore not one of our prioritised requirements. Our focus with the web solution is to give the user a tool, which enables them to track their nutrition and food-related emissions.

The set value has some issues as many variables affect how many calories a person should intake per day, such as gender, height, weight and their desired weight. Nevertheless, we decided to implement the graph with the intention of visualising the connection between food intake and

co2 emissions. The set value has some issues as many variables affect how many calories a person should intake per day, such as gender, height, weight, and their desired weight. Nevertheless, we decided to implement the graph with the intention of visualising the connection between food intake and CO2 emissions. The feature allows for further coding regarding user inputted goals but necessitates implementing a backend route that can handle this specific user input. The comparisons are especially important so the user is not merely given a lot of information through text, but can understand it through comparison to the general public.

R5 concerned transparency in our web solution. This is based on the apps from related works which have pages, where the users can see how their data has been collected and calculated. Our findings from the WOZ approach and the research papers we had read whilst exploring the problem area also stressed the importance of transparency. As explained in the introduction, young Danes feel lost when they try to navigate climate friendly habits. One of the reasons for this is that there are different ways to perceive whether something is climate friendly or not, depending on which factors are taken into account. Nestle also highlights how some food companies misuse the interest from consumers in climate-friendly and nutritious food in advertisements.(ref til nestle) Her research strongly emphasises the need for transparency to the users. Whilst we tested the prototype we came up with the idea of simply having a page in our web solution dedicated to information about our data and especially CO2 calculations. We would rather have all the information about transparency in one place instead of having it scattered around the web solution, so the user can read it if they want but it is not something they have to consider all the time. Therefore, we solved this requirement by simply having an easily accessible html page called “Information”, which is accessible from all pages in the web solution.

5.5 Further work

R6 focused on the users setting individual goals. The system should allow users to set personalised goals for their macro intake and CO2 emissions. This feature was inspired by apps like MyFitnessPal and Floop. These apps emphasise the importance of the user being in focus and it being customisable. However, the implementation of this requirement has been delayed due to a combination of factors. It is not a part of the first requirements, which are crucial and focuses on delivering a functional core web solution. This approach to constructing ensures the stability and usability of our program before we add optional requirements like individual goal setting. While valuable, it is planned as part of future updates when the core system is fully established.

R7 was “Tips on Gauging the Weight of Ingredients” and this requirement was inspired by the Floop app and would ideally provide users with information about the weight of standard ingredients. By offering this data, users would be able to estimate ingredient quantities more easily, reducing the need to weigh everything manually to achieve maximal accuracy through the web

solution. Which may eventually make the web solution convenient in everyday life. However, the implementation of this feature has been delayed due to its complexity. This feature has been deprioritised for now, but it remains a valuable addition for future updates.

R8 concerns what we call a “history” element. This requirement, inspired by popular apps like MyFitnessPal and Floop, involves enabling the web solution to generate detailed reports for users about their macros and CO2 emissions over an extended period. Such a feature is crucial for providing users with the ability to review their historical data, assess trends, and track progress toward their goals. By offering a comprehensive view of their past behavior, the history element empowers users to make more informed decisions, identify patterns, and stay motivated by recognising how far they’ve come. Despite its potential to significantly enhance user engagement and satisfaction, the implementation of this feature has been deferred.

Our primary focus has been on ensuring the web solution’s foundational functionality, prioritising the first five core requirements. These foundational features have been critical for delivering a stable and testable web solution, allowing us to meet the project’s initial goals. However, the history element represents an exciting avenue for future development. Its integration would not only provide additional value by transforming the web solution from a basic utility into a more dynamic and insightful tool but also align with user expectations set by similar successful apps in the market. In future updates, this feature could be a key differentiator, showcasing our commitment to long-term user support and continuous improvement.

6 Conclusion

Our project sought to answer: How can a web solution provide a tool to help young Danes track and improve their nutritional intake while reducing their CO₂ emissions? We developed a tool that calculates macros and CO₂ emissions in food, calculating it with input (food and amount of food) from the user. From there the results needed to be displayed to the user through simple visualisations. We did this by showing the user the raw data of their food items on the search page, through charts on the frontpage and finally through an emoji corresponding to the results of the user input on both pages, ultimately fulfilling the problem statement by creating a web tool that enhances their environmental and nutritional awareness. Through the process of creating the web solution, our problem statement raised several challenges which emphasised the complexity of the problem area. Our reliance on “Klimadatabasen” ensured scientific grounding while also revealing limitations regarding the complexity of food systems, such as but not limited to production and transportation.

Through the iterative development of our web solution, the project managed to fulfill what we deemed as the most crucial requirements for the core functionalities which are user login, calculations regarding nutrition and emissions and finally the display and comparison element of the users’ food data. These requirements were solved successfully, however there is room for further developments. The last three requirements are desired features for the ideal web solution, but not necessary for the functionality. These desired features concerns customisable goal setting, unit flexibility and a history element. We believe that if these features were implemented into our web solution, it could enhance the user experience.

The web solution offers a tool for young Danes between 18-29 to track their nutritional intake while being informed of their CO₂ emissions. While giving them data that makes it easier for them to take action upon their own health as well as the planets health. Our web solution can play a critical role in helping young Danes between 18-29 in balancing sustainability and nutrition. Our web solution bridges nutrition with environmental impact, offering a concrete pathway for users to take small, but measurable steps towards reducing their CO₂ emissions while staying on top of their nutrition. Our web solution gives young Danes a tool, which they can use to track and improve their nutritional intake while reducing their CO₂ emissions.

7 Reflection

For this project the group has been made on the premise of us having different backgrounds. Our bachelors range from communication, art and technology, techno-anthropology to history. This proved to be interesting for the project work, as we all had different things to contribute with. It was especially useful in the brainstorming phase of the project, as we each could provide a different perspective during our idea generation. However, it also required us to align expectations and roles to effectively collaborate throughout the process(Holgaard 2021, p.88).

Expectations, should have talked about it more than just once as the whole project just an iterative process and we constantly learned new things about how a project at this faculty should look like. An issue we experienced was how the entire group was did not fully agree on how the different elements should be done, resulting in group members working independently on tasks without consulting the group. This lead to creation of sections that ultimately could not be used in the project, requiring additional revisions and spending time on corrections that could have been avoided. Another significant challenge was during the coding phase. Not all members had computers that could run the programs we use for coding. We quickly realised that coding essential part of the learning process and essentially was trial and error, where we each had to try different things on our computers without knowing the solution before hand. It was quite frustrating when we could not split up and try different solutions. It led to us wasting time, where the solution to our coding issue might have been obvious, but could not be tested due to computer equipment. Since we were not in a position to quickly translate ideas into functional code and evaluate whether they were indeed correct, it lead to delays and inefficiencies in our workflow.

Despite the challenges, we learned valuable lessons about the importance of clear communication, ongoing expectation alignment and clearly defining roles within the group(Holgaard 2021, 97). We recognised the need for better planning and resource allocation, particularly when technical limitations arise.

Bibliography

Andresen, Lars Nøhr. 2023. *Er plantebaserede alternativer mere forarbejdede end kød?*

BBC. 2024a. *Abstraction*. Last accessed 19 December 2024. <https://www.bbc.co.uk/bitesize/guides/ztttrcdm/revision/1>.

———. 2024b. *Algorithm Design*. Last accessed 19 December 2024. <https://www.bbc.co.uk/bitesize/guides/zpp49j6/revision/1>.

———. 2024c. *Decomposition*. Last accessed 19 December 2024. <https://www.bbc.co.uk/bitesize/guides/zqqfyrd/revision/1>.

———. 2024d. *Introduction to computational thinking*. Last accessed 04 December 2024. <https://www.bbc.co.uk/bitesize/guides/zp92mp3/revision/1>.

———. 2024e. *Pattern Recognition*. Last accessed 19 December 2024. <https://www.bbc.co.uk/bitesize/guides/zxxbgk7/revision/1>.

Benvenuti, Luca, and Alberto De Santis. 2020. “Making a Sustainable Diet Acceptable: An Emerging Programming Model With Applications to Schools and Nursing Homes Menus.” *Frontiers in Nutrition* 7. ISSN: 2296-861X. <https://doi.org/10.3389/fnut.2020.562833>. <https://www.frontiersin.org/journals/nutrition/articles/10.3389/fnut.2020.562833>.

Benyon, David. 2010. *Designing Interactive Systems*. 2nd. Essex: Pearson Education Limited.

Bernsen, Niels Ole, Hans Dybkjær, and Laila Dybkjær. 1994. “Wizard of oz prototyping: How and when.” *Proc. CCI Working Papers Cognit. Sci./HCI, Roskilde, Denmark*.

Clara Laurine Berg-Jensen, Mikael Bellers Madsen og. 2021. *Unges madvaner under forandring*. https://concito.dk/files/media/document/SOMA%20rapport_0.pdf.

CONCITO. 2024a. *Den store klimadatabase*. <https://denstoreklimadatabase.dk/>.

———. 2024b. *Den store Klimadatabase*. Accessed: 2024-12-08. <https://denstoreklimadatabase.dk/baggrund>.

———. 2024c. *Den store klimadatabase, baggrundsrapport 2.1*. Accessed: 2024-04-08. <https://concito.dk/files/media/document/Den%20Store%20Klimadatabase%20version%201.2%20-%20baggrundsrapport.pdf>.

- CONCITO. 2024d. *Denmark's green think tank CONCITO*. Accessed: 2024-12-08. <https://concito.dk/denmarks-green-think-tank-concito>.
- Dictionary, Oxford English. 2024. *Nutrition*. Last accessed 04 December 2024. <https://www.oed.com/search/dictionary/?scope=Entries&q=nutrition>.
- Eaternity. n.d. *Climate Change and Foodprint*. <https://eaternity.org/foodprint/climate-change>. Accessed: 2024-12-10.
- Foundation, Python Software. 2024. *sqlite3 — DB-API 2.0 interface for SQLite databases*. Accessed: 2024-12-08. <https://docs.python.org/3/library/sqlite3.html>.
- Frega, Romeo, Jose Guerra Lanfranco, Sam De Greve, Sara Bernardini, Perrine Geniez, Nils Grede, Martin Bloem, and Saskia de Pee. 2012. “What Linear Programming Contributes: World Food Programme Experience with the “Cost of the Diet” Tool.” *Food and Nutrition Bulletin* 33 (3_suppl2): S228–S234. <https://doi.org/10.1177/15648265120333S212>.
- GeeksforGeeks. 2024. *Flask URL Helper Function — Flask url_for*. Accessed: 2024-12-08. https://www.geeksforgeeks.org/flask-url-helper-function-flask-url_for/.
- Gioia, Siena, Irma M Vlasac, Demsina Babazadeh, Noah L Fryou, Elizabeth Do, Jessica Love, Rebecca Robbins, Hassan S Dashti, and Jacqueline M Lane. 2023. “Mobile Apps for Dietary and Food Timing Assessment: Evaluation for Use in Clinical Research.” *JMIR Form Res* 7.
- Al-Hassan, Soondus M. 2016. *Validating the MyFitnessPal mobile application in assessing dietary intake against the 7-day food diary: a randomised cross-over pilot study*. <http://hdl.handle.net/10034/620469>.
- Holgaard, Jette Egelund. 2021. *An introduction to problem-based learning in higher education*. [in eng]. 1. edition. Frederiksberg: Samfundslitteratur. ISBN: 9788759338957.
- Irz, Xavier, Jørgen Dejmård Jensen, Pascal Leroy, Vincent Réquillart, and Louis-Georges Soler. 2019. “Promoting climate-friendly diets: What should we tell consumers in Denmark, Finland and France?” *Environmental Science Policy* 99:169–177. ISSN: 1462-9011. <https://doi.org/https://doi.org/10.1016/j.envsci.2019.05.006>. <https://www.sciencedirect.com/science/article/pii/S1462901119302734>.
- Lawo, Dennis, Thomas Neifer, Margarita Esau, and Gunnar Stevens. 2021. “Buying the ‘Right’ Thing: Designing Food Recommender Systems with Critical Consumers.” In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. CHI '21. Yokohama, Japan: Association for Computing Machinery. ISBN: 9781450380966. <https://doi.org/10.1145/3411764.3445264>. <https://doi.org/10.1145/3411764.3445264>.

- Madkulturen. 2019. *Bæredygtighed og Madkultur 2019*. Technical report. Last accessed 04 December 2024. Madkulturen. <https://www.madkulturen.dk/wp-content/uploads/2021/09/Baeredygtighed-og-madkultur-2019.pdf>.
- Masino, Talia, Patricia Eustachio Colombo, Kia Reis, Inge Tetens, and Alexandr Parlesak. 2023. “Climate-friendly, health-promoting, and culturally acceptable diets for German adult omnivores, pescatarians, vegetarians, and vegans – a linear programming approach.” *Nutrition* 109:111977. ISSN: 0899-9007. <https://doi.org/https://doi.org/10.1016/j.nut.2023.111977>. <https://www.sciencedirect.com/science/article/pii/S0899900723000072>.
- Maulsby, David, Saul Greenberg, and Richard Mander. 1993. *Prototyping an intelligent agent through Wizard of Oz*. New York, NY, USA. <https://doi.org/10.1145/169059.169215>. <https://doi.org/10.1145/169059.169215>.
- Minter, Michael, and Charlotte Louise Jensen. 2024. *Danmarks globale forbrugsudledninger*. Last accessed 04 December 2024. <https://concito.dk/udgivelser/danmarks-globale-forbrugsudledninger>.
- Miriam, Meister. 2024. *Gennemsnitsdanskere spiser 52 kilo kød om året*. <https://www.food.dtu.dk/nyheder/nyhed?id=1481037a-8136-4db4-9d79-6767e6dc1592>.
- Nestle, Marion. 2018. *Unsavory Truth: How Food Companies Skew the Science of What We Eat*. New York, NY: Basic Books.
- OpenCourseWare - Harvard. 2020. *This is CS50*. Accessed: 2024-12-08. <https://cs50.harvard.edu/x/2020/notes/7/>.
- . 2024. *This is CS50*. Accessed: 2024-12-08. <https://cs50.harvard.edu/x/2024/notes/9/%5C#flask-and-sql>.
- Pallets Projects. 2024a. *Flask Documentation*. Accessed: 2024-12-08. <https://flask.palletsprojects.com/en/stable/reqcontext/>.
- . 2024b. *Jinja2 Template Documentation - Conditional Statements*. Accessed: 2024-12-08. <https://jinja.palletsprojects.com/en/stable/templates/#if>.
- Springmann, Marco, H. Charles J. Godfray, Mike Rayner, and Peter Scarborough. 2016. “Analysis and valuation of the health and climate change cobenefits of dietary change.” *Proceedings of the National Academy of Sciences* 113 (15): 4146–4151. <https://doi.org/10.1073/pnas.1523119113>. <https://doi.org/10.1073/pnas.1523119113>.
- Sundhedsstyrelsen. 2021. *Danskernes Sundhed*. <https://www.sst.dk/-/media/Udgivelser/2022/Sundhedsprofil/Sundhedsprofilen.ashx>.

- Thingsted, Anne Sophie. 2021. *Har unge virkelig de mest klimavenlige madvaner?* <https://videnskab.dk/kultur-samfund/har-unge-virkelig-de-mest-klimavenlige-madvaner/>.
- Turner, Angela M. 2019. *Use of Smartphone Application to Facilitate Weight Loss and Promote Accountability in Obese and Overweight Patients*.
- Union, Den Europæiske. 2021. *Køb grønt: dine valg som forbruger tæller*. https://youth.europa.eu/get-involved/sustainable-development/buying-green-make-your-consumer-choices-count_da.
- W3Schools. 2024a. *CSS Tutorial*. Accessed: 2024-12-08. <https://www.w3schools.com/css/default.asp>.
- . 2024b. *HTML Tutorial*. Accessed: 2024-12-08. <https://www.w3schools.com/html/default.asp>.
- . 2024c. *JavaScript Google Charts Tutorial*. Accessed: 2024-12-08. https://www.w3schools.com/js/js_graphics_google_chart.asp.
- . 2024d. *W3Schools Online Web Tutorials*. Accessed: 2024-12-08. <https://www.w3schools.com/>.
- Willett, Walter. 2020. “Food in the Anthropocene: the EAT–Lancet Commission on healthy diets from sustainable food systems.” *The Lancet* 393:447–492. [https://doi.org/10.1016/S0140-6736\(18\)31788-4](https://doi.org/10.1016/S0140-6736(18)31788-4).